Comparative Evaluation of Machine Learning Classifiers for Extreme Rainfall at the Kathmandu Airport Grid Cell, Nepal

Bishow Gurung
College ID: 260005 | Coventry University ID: 17624457
MSc Data Science and Computational Intelligence
Softwarica College of IT & E-Commerce, Kathmandu, Nepal
Email: 260005@softwarica.edu.np | Module: STW7072CEM Machine Learning

***Abstract*—Heavy monsoon rain floods the Kathmandu Valley almost every year. Dry and light-rain days outnumber extreme cloudbursts by nineteen to one in gridded weather records, and that skew breaks standard forecast models. Here,** 10,958 **raw daily weather records** (1995–2024; 10,951 **post-lag rows) from the NASA POWER and MERRA-2 archive at Tribhuvan International Airport are divided at** 31 **December** 2018 **into** 8,759 **training days (**439 **storms** $\geq$ 13.74 **mm/day, the training** 95**th percentile) and** 2,192 **unseen test days (**2019–2024**;** 164 **storms). Seven Principal Component Analysis (PCA) axes keep** 91.46% **of predictor variance. Fast Independent Component Analysis (FastICA) pulls out non-Gaussian rain spikes, and three-cluster K-Means groups** 94.99% (417 **of** 439**) of training storms into one active monsoon regime. On the** 2019–2024 **holdout set, five models are benchmarked: Logistic Regression, Decision Tree (CART), Radial Basis Function Support Vector Machine (RBF-SVM), Random Forest, and k-Nearest Neighbours (k-NN). Random Forest leads in harmonic F**1 (0.4385) **and Precision-Recall AUC** (0.5517) **with** 90.24% **storm recall, whereas balanced Logistic Regression flags** 96.34% **of test storms** (158 **of** 164**). Unweighted k-NN fails. Despite** 93.16% **accuracy, it misses** 71.34% **of storms. When the Random Forest cutoff drops to** 0.30 **under a** 15**-to-**1 **missed-flood loss, test recall climbs to** 98.17% (161 **of** 164**) and catches the** 24–28 **September** 2024 **Kathmandu flood a day early.**

***Index Terms*—Cost-sensitive threshold tuning, Decision Tree, extreme rainfall classification, feature importance, imbalanced weather data, Kathmandu Valley, K-Means clustering, Logistic Regression, machine learning comparison, MERRA-2 reanalysis, monsoon flood warning, Principal Component Analysis, Random Forest, Support Vector Machine, Tribhuvan International Airport.**

I. Introduction

A. Background and Motivation

Active monsoon troughs and late-September depressions bring severe flooding to central Nepal [1, 2, 3]. High ridges at Shivapuri, Phulchowki, and Chandragiri (>2,500 m) ring the Kathmandu Valley floor (1,300–1,400 m) and funnel all Bagmati and Bishnumati runoff through one narrow rock outlet at Chobhar [4, 5]. Rapid paving over the last two decades has sealed most natural floodplains. Even 14 to 25 mm of daily rain now swamps street drains in a few hours [5, 6, 7, 8]. Between 24 and 28 September 2024, five straight days of torrential rain reached 93.26 mm/day on the Tribhuvan International Airport grid cell, inundated riverside settlements, shut down airport operations, and claimed over 240 lives across Nepal [6, 7].

Physics-based weather models track regional South Asian monsoon flow well [9, 10, 11]. Even so, they demand heavy computing power and blur local valley cloudbursts across wide grid boxes [3, 12], whereas statistical classifiers run in seconds by linking daily humidity, temperature, air pressure, sunlight, and past rain directly to storm probabilities [1, 2, 12]. Three obstacles complicate the task. Extreme rain ($\geq$ 13.74 mm/day) falls on only 5.01% of training days (a 1-to-19 ratio) and biases standard models toward dry forecasts [2, 3], while multi-day monsoon wet spells cause severe data leakage under random train-test splits [1, 13]. Missed floods also cost far more than false alarms [3, 6].

B. Research Aim and Objectives

This study tests unsupervised weather-grouping methods and five supervised classifiers for extreme daily rainfall detection at the Kathmandu Airport grid cell. Three objectives structure the experiments:

- **RO1 (Data Pipeline and Unsupervised Regimes):** Build a leak-free 30-year daily dataset (1995–2024; 10,951 post-lag rows and 22 predictors) from NASA POWER and MERRA-2 records with a strict split at 31 December 2018, then run PCA, FastICA, and K-Means to measure storm concentration across seasonal weather regimes.
- **RO2 (Classifier Comparison and Feature Drivers):** Train Logistic Regression, Decision Tree (CART), RBF-SVM, Random Forest, and k-NN on the 1995–2018 record. Compare their 2019–2024 test recall, feature rankings, Brier calibration, 7-component PCA skill, and runtime speed.
- **RO3 (Cost-Sensitive Thresholds and Operational Tests):** Tune warning cutoffs under a 15-to-1 missed-flood penalty and test 24-hour advance forecasts, wet-day deluges ($\geq$ 22.75 mm/day), and the September 2024 Kathmandu flood.

II. Literature Review

A. Himalayan Extreme Rainfall Trends

Total wet days per year in Kathmandu have stayed nearly flat over the past three decades. Peak downpours tell a different story [4, 5, 8]. In valley gauge series, Chaulagain *et al.* [4] and Khanal *et al.* [5] linked steeper 95th-percentile rain totals to warmer urban land surfaces and stronger moisture transport from the Bay of Bengal, while Phuyal [8] showed that almost 90% of heavy rain days occur between June and September. Elsewhere in the Himalaya, Wani *et al.* [9], Pujara and Paudel [10], and Bashyal *et al.* [11] found that linear time-series models miss abrupt mountain cloudbursts. Deep recurrent and transformer models track nonlinear shifts better yet demand dense station networks and heavy compute [11, 12]; gridded reanalysis products such as NASA POWER and MERRA-2 bridge those gauge gaps across long multi-decade windows [13].

B. Limitations of Continuous Rainfall Regression

Most recent rainfall studies treat storm forecasting as continuous millimetre regression. That choice fails on skewed daily records. In a 10-year Kathmandu study (2015–2025), Poudyal *et al.* [1] evaluated Random Forest, Support Vector Regression, k-NN, and recurrent networks via root-mean-square error (RMSE) and mean absolute error (MAE), where equal weighting of dry and wet days caused their top regressor to compress real peaks above 17 mm/day down below 7 mm/day [1, 3]. An even starker failure occurred in Nurmayati's [2] study of 52,703 ten-minute weather rows in South Tangerang, Indonesia. Because more than 90% of intervals recorded 0 mm, Random Forest, SVM, and k-NN achieved tiny mean errors (< 0.10 mm) by outputting near-zero rain all day—which gave negative R^2 values (-0.062 to -14.378) while random 10-fold splits leaked multi-hour storms across folds. Large neural forecast models like Pangu-Weather [14] and GraphCast [15] likewise damp local grid-cell extremes under average loss functions [3], whereas binary classification above a fixed percentile cutoff targets the tail directly [16].

C. Unsupervised Clustering and Imbalanced Classification

Unsupervised regime analysis helps structure correlated weather tables before supervised training. Zhang, Liu, and Chen [17] combined PCA with K-Means++ to sort humidity, temperature, and pressure records into seasonal rainfall regimes, while FastICA separates spiky non-Gaussian rain pulses from smooth annual waves. On skewed flood datasets, benchmarks by Wu *et al.* [18], Liu *et al.* [19], and Toufique *et al.* [20] show that class-weighted models and bagged tree ensembles detect far more rare storms than unweighted neighbour lookup when evaluated with date splits and Precision-Recall curves [2, 3].

D. Identified Research Gaps

Three specific gaps remain in the 2022–2026 Kathmandu literature. First (**RO1**), no prior work has run PCA, FastICA, and K-Means on a leak-free 30-year MERRA-2 archive to quantify how strongly Kathmandu storms concentrate inside one monsoon cluster. Second (**RO2**), existing local papers rely on regression loss [1] or random cross-validation [2] instead of benchmarking linear, single-tree, kernel, ensemble, and nearest-neighbour classifiers on unseen storm recall, Brier calibration, and 7-component PCA compression. Third (**RO3**), no study in Nepal has calibrated warning cutoffs against a 15-to-1 missed-flood cost or tested 24-hour advance warnings on the September 2024 Kathmandu flood. Those three gaps shape the methodology below.

III. Problem and Dataset Description

A. Problem Definition and Significance

Dry days dominate the training loss. Continuous rainfall regressors therefore pull storm peaks down toward the dry average, whereas binary classification targets fixed 95th-percentile cutoffs directly ($P_{95} = 13.74$ mm/day across all days and $R95p = 22.75$ mm/day across wet days ≥ 1.0 mm). Each classifier takes 22 daily weather and lagged-rainfall features on day t and outputs a storm probability $\hat{p}_t \in [0, 1]$. Early probability alerts at Tribhuvan International Airport and along the Bagmati river corridor give forecasters and city crews time to clear drains before flooding starts.

B. Study Site and 30-Year Weather Dataset

1) Kathmandu Airport Grid Cell and Seasonality

The NASA Langley Research Center POWER archive (GMAO MERRA-2 reanalysis) supplies 30 years of daily weather records (1 January 1995 to 31 December 2024; 10,958 raw days) for the $0.5° \times 0.625°$ grid cell at Tribhuvan International Airport ($27.697°\text{N}, 85.359°\text{E}$; 1,269.39 m elevation) [4, 13]. Steep mountain walls surround the valley floor. Peaks at Shivapuri (2,732 m) and Phulchowki (2,762 m) trap moist air over the central plateau and force strong upward cloud growth. Four seasons govern the yearly cycle: cold, dry winter weather from December to February under continental westerlies; hot pre-monsoon days from March to May with strong sunshine and afternoon thunder; the wet South Asian summer monsoon from June to September, when Bay of Bengal depressions pump saturated air into the basin; and cool post-monsoon weather in October and November. In Fig. 1, 88.7% of all extreme rain days fall between June and September, and July–August alone account for nearly two-thirds of historical cloudbursts.

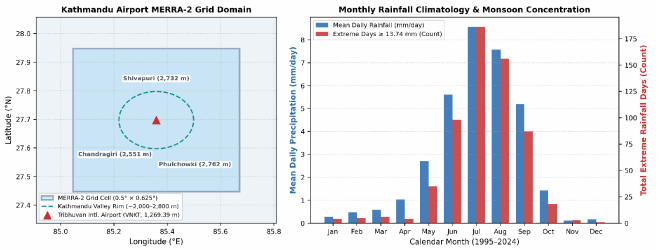


Fig. 1. Study domain for the NASA POWER MERRA-2 grid cell centred at Tribhuvan International Airport within the Kathmandu Valley rim (left), alongside the 1995–2024 monthly mean rainfall and extreme event counts (right).

2) Chronological Train-Test Split and Data Quality Audit

Every calendar date from 1995 through 2024 is present. Automated checks in `data_processing.py` confirmed zero missing dates, zero duplicate rows, zero blank entries, and zero negative rain values across all 10,958 raw days. Because the 7-day rolling rainfall totals need one week of prior history, the script trims the first 7 dry winter days of January 1995 (1–7 January 1995; maximum rain 0.16 mm) and retains $N = 10,951$ complete rows. A strict date split at 31 December 2018 divides those rows into an 80% training set (1995–2018; 8,759 days) and a 20% holdout test set (2019–2024; 2,192 days; Table I).

TABLE I
CHRONOLOGICAL DATASET PARTITIONS, SAMPLE SIZES, AND EXTREME RAINFALL COUNTS

Dataset Partition	Date Range	All-Day Threshold (13.74 mm)	Wet-Day Threshold (22.75 mm)
Raw Archive	1995-01-01 to 2024-12-31	10,958 days (603 extremes)	10,958 days (212 extremes)
Lag Trim Window	1995-01-01 to 1995-01-07	7 winter days (0 extremes)	7 winter days (0 extremes)
Complete Modelling Set	1995-01-08 to 2024-12-31	10,951 days (603 extremes, 5.51%)	10,951 days (212 extremes, 1.94%)
Training Set (80%)	**1995-01-08 to 2018-12-31**	**8,759 days (439 extremes, 5.01%)**	**8,759 days (148 extremes, 1.69%)**
Holdout Test Set (20%)	**2019-01-01 to 2024-12-31**	**2,192 days (164 extremes, 7.48%)**	**2,192 days (64 extremes, 2.92%)**

Summary statistics for all 13 raw training variables appear in Table II. One archive anomaly required special care: 10 m wind direction (`WD10M`) logs normal compass degrees (1.20° to 359.90°) until 30 December 2012, then switches to negative calm-wind numbers (-8994.0 to -5404.0) from 31 December 2012 onward, so the preprocessing script captures that state with a binary flag (`wind_calm_flag`) and excludes raw `WD10M` from continuous z-score scaling.

TABLE II
RAW METEOROLOGICAL VARIABLES AND TRAINING SUMMARY STATISTICS (1995–2018, 8,759 DAYS)

Variable	Units	Physical Weather Role	Train Mean ± SD	Min / Max
PRECTOTCORR	mm/day	Daily total rainfall (target source)	2.59 ± 5.72	0.00 / 60.01
T2M	°C	Mean 2m air temperature	19.22 ± 4.98	4.11 / 30.11
T2M_MAX	°C	Maximum daily 2m temperature	24.72 ± 4.64	9.93 / 37.30
T2M_MIN	°C	Minimum daily 2m temperature	15.10 ± 5.08	-0.12 / 24.91
T2MDEW	°C	Dew point temperature at 2m	8.78 ± 9.35	-16.63 / 22.26
RH2M	%	Relative humidity at 2m	57.31 ± 23.06	5.43 / 96.45
QV2M	g/kg	Specific humidity (water vapour)	9.67 ± 5.38	0.97 / 19.56
PS	kPa	Surface air pressure	87.27 ± 0.40	85.92 / 88.20
WS10M	m/s	Mean 10m wind speed	2.17 ± 0.52	0.86 / 4.65
WS10M_MAX	m/s	Peak daily 10m wind gust	4.49 ± 1.02	1.72 / 9.55
WD10M*	Degrees	10m wind direction (non-calm days)	217.8 ± 43.7	1.20 / 359.9
ALLSKY_SW	MJ/m^2/d	Downward solar radiation	17.60 ± 4.69	2.21 / 30.54
TS	°C	Surface skin temperature	18.46 ± 6.43	2.60 / 31.46

*Note: Post-2012 negative calm-wind codes in WD10M are captured via a binary calm-wind indicator.

C. Target Thresholds and Operational Classification Tasks

1) Extreme Rainfall Thresholds (13.74 mm and 22.75 mm)

Both storm cutoffs come strictly from the 1995–2018 training partition [3, 4]. The all-day 95th percentile sits at $P_{95} = 13.74$ mm/day (439 training storms at 5.01%; 164 holdout storms at 7.48%), whereas the wet-day 95th percentile across days with ≥ 1.0 mm of rain sits at $R95p = 22.75$ mm/day (148 training deluges at 1.69%; 64 holdout deluges at 2.92%; Fig. 2).

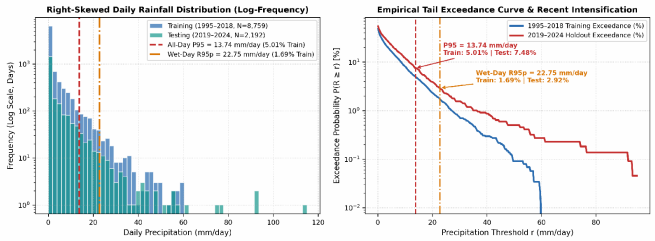


Fig. 2. Daily rainfall frequency distribution (left) and exceedance probability curves for the 1995–2018 training period versus the 2019–2024 holdout period (right) at the 13.74 mm/day and 22.75 mm/day thresholds.

2) Operational Classification Tasks (Tasks A, B, and C)

Forecasters need both same-day diagnosis and 24-hour advance warning. Three tasks therefore run on the 2019–2024 holdout set:

- **Task A (Same-Day Atmospheric Diagnosis, 13.74 mm/day):** Pairs 15 day-t non-rainfall weather variables with 5 lagged rainfall predictors ($t-1$ to $t-7$) and 2 seasonal waves to classify same-day storms.
- **Task B (Strict 24-Hour Advance Forecast, 13.74 mm/day):** Shifts all 15 weather variables back to day $t-1$. Every input comes from at least 24 hours earlier.
- **Task C (Severe Wet-Day Deluge Detection, 22.75 mm/day):** Flags severe downpours at or above the wet-day 95th percentile ($R95p = 22.75$ mm/day).

IV. Methods and Experimental Setup

A. Experimental Setup, Data Pre-Processing, and Feature Extraction

Three Python 3 scripts run all experiments. Built on `scikit-learn`, `NumPy`, `pandas`, and `SciPy`, `data_processing.py`, `train_and_evaluate.py`, and `run_experiments.py` execute the six-stage workflow and 5-fold expanding-window time-series validation schedule in Fig. 3 after stripping same-day rainfall (`PRECTOTCORR` on day t) from every feature table.

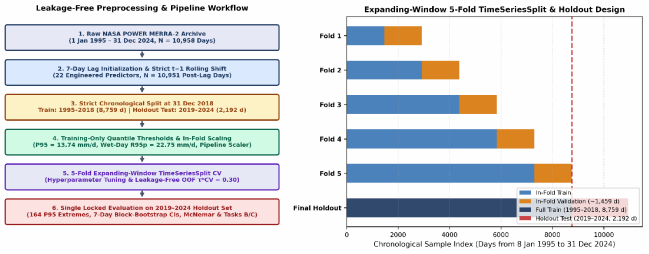


Fig. 3. Leak-free data preparation and evaluation workflow (left) and the 5-fold expanding-window time-series validation schedule within the 1995–2018 training set (right).

Table III lists the 22 engineered predictors. Four surface indicators capture cloud and pressure changes: daily temperature range (`T2M_MAX` minus `T2M_MIN`, which narrows when thick monsoon clouds block afternoon sun and trap nighttime warmth), dew-point spread (`T2M` minus `T2MDEW`, which measures the gap to 100% air saturation), 24-hour pressure change (today's minus yesterday's surface pressure), and the binary calm-wind flag. Five past-rainfall predictors (`precip_lag1`, `precip_lag2`, `precip_lag3`, `precip_roll3`, and `precip_roll7`) use a strict 1-day backward shift so the 3-day and 7-day sums cover only days $t-1$ through $t-7$. Sine and cosine waves of the calendar day (`sin_doy`, `cos_doy`) track the annual cycle. Task B lags all 15 weather predictors to day $t-1$. Scale-sensitive models (PCA, FastICA, K-Means, Logistic Regression, RBF-SVM, and k-NN) standardise continuous columns inside each training fold to zero mean and unit variance using only that fold's training statistics.

TABLE III
INVENTORY OF THE 22 ENGINEERED WEATHER PREDICTORS

#	Predictor Name	Calculation Method and Units	Physical Weather Meaning
1	T2M	Daily mean 2m air temperature (°C)	Warmth governing moisture capacity
2	T2M_MAX	Daily maximum 2m temperature (°C)	Daytime surface heating peak
3	T2M_MIN	Daily minimum 2m temperature (°C)	Nighttime cloud heat trapping
4	temp_diurnal_range	Max minus min daily temperature (°C)	Cloud cooling (narrows on rainy days)
5	T2MDEW	Dew point temperature at 2m (°C)	Condensation temperature threshold
6	dew_point_depression	Air temperature minus dew point (°C)	Near-surface air saturation gap
7	RH2M	Relative humidity at 2m (%)	Near-surface moisture saturation
8	QV2M	Specific humidity at 2m (g/kg)	Water vapour fuel for storm clouds
9	PS	Surface atmospheric pressure (kPa)	Monsoon low-pressure trough depth
10	pressure_tendency	Today minus yesterday pressure (kPa)	24-hour barometric rise or fall
11	WS10M	Mean 10m wind speed (m/s)	Valley moisture transport speed
12	WS10M_MAX	Maximum daily 10m wind gust (m/s)	Storm downdraught gust strength
13	ALLSKY_SW	Downward solar radiation (MJ/m^2/d)	Sunlight blocked by storm clouds
14	TS	Surface skin temperature (°C)	Land-surface heating state
15	wind_calm_flag	Binary flag for negative wind code (0/1)	Calm or variable valley wind state
16	precip_lag1	Rainfall 1 day ago (mm/day)	Yesterday's rainfall persistence
17	precip_lag2	Rainfall 2 days ago (mm/day)	48-hour storm track memory
18	precip_lag3	Rainfall 3 days ago (mm/day)	72-hour wet spell persistence
19	precip_roll3	Total rainfall over past 3 days (mm)	Short-term soil and air wetting
20	precip_roll7	Total rainfall over past 7 days (mm)	Weekly catchment moisture build-up
21	sin_doy	Sine wave of calendar day of year	Smooth seasonal progression (spring/autumn)
22	cos_doy	Cosine wave of calendar day of year	Smooth winter-to-monsoon cycle

B. Unsupervised Pattern Analysis and Clustering Methods (RO1)

1) Principal Component Analysis (PCA)

Three columns in Table III are exact sums or differences of other columns. Because daily temperature range, dew-point spread, and 3-day rolling rain overlap linearly with their parent variables, PCA removes that redundancy under RO1 and builds an orthogonal subspace for the RO2 ablation test [17]. A 22-component PCA model (seed 42) fits strictly on the standardised $8,759 \times 22$ training matrix without access to storm labels or 2019–2024 holdout rows. Seven axes ($m = 7$) pass the 90% variance rule at 91.46% cumulative variance. Both training and holdout tables then project onto those 7 fixed training axes.

2) Fast Independent Component Analysis (FastICA)

Cloudbursts arrive as sharp spikes rather than smooth bell-shaped waves. While PCA groups variables by broad seasonal variance, FastICA runs on the 7-component whitened training table (7 sources, log-cosh contrast, 500 maximum iterations, seed 42) to separate non-Gaussian rainfall pulses from the post-2012 wind-direction code shift under RO1. The algorithm rotates the whitened axes to maximise excess kurtosis along each extracted source. This diagnostic checks whether multi-day rain bursts isolate on a single high-kurtosis axis and shows why unitless rotated sources are harder for duty forecasters to interpret than physical thresholds.

3) K-Means Clustering

K-Means clustering partitions the 8,759 unlabelled training days into K seasonal regimes under RO1 [17]. Across candidate counts $K \in \{2,3,4,5,6\}$ (10 random restarts, seed 42), the algorithm assigns each standardised training day to the nearest cluster centre by squared Euclidean distance. Both the full-sample Silhouette score ($N = 8,759$) and the Davies–Bouldin index score each K. Those two metrics select $K = 3$ objectively. After convergence, the script converts the cluster centres back into physical units (g/kg, °C, kPa, mm) and cross-tabulates the clusters against historical storm labels to measure storm concentration inside the monsoon regime.

C. Supervised Classification Algorithms (RO2 and RO3)

1) Logistic Regression

Logistic Regression provides the linear baseline for RO2 and the high-sensitivity moisture screen for RO3 [18]. Each validation fold runs z-score scaling inside a pipeline before fitting L2-penalised Logistic Regression (LBFGS solver, 1,000 maximum iterations, seed 42) so test statistics never leak backward. Non-extreme days outnumber training storms by 19 to 1 (8,320 vs. 439). Balanced class weights therefore multiply storm errors by $w_1 = 9.98$ and non-extreme errors by $w_0 = 0.53$ during training. Five-fold expanding time-series grid search over $C \in \{0.01, 0.1, 1.0, 10.0\}$ selects $C = 0.01$ for Task A ($C = 0.1$ for Tasks B and C), where strong L2 shrinkage keeps correlated humidity variables from taking unstable weights.

2) Decision Tree (CART)

Decision trees partition weather predictors through hierarchical threshold rules rather than a single linear equation [16]. Here, the Classification and Regression Tree (CART) algorithm serves as an interpretable rule baseline under RO2 to test how greedy Gini impurity splitting behaves when yesterday's rain dominates multi-day monsoon wet spells. Monotone feature scaling does not alter tree split points. CART therefore trains directly on the unscaled physical matrix (balanced class weights, seed 42) so every node threshold stays in native units (mm/day, °C, kPa), while a 45-point expanding 5-fold time-series search over depths $\{3,4,5,6,8\}$, minimum split sizes $\{10,20,50\}$, and minimum leaf sizes $\{5,10,20\}$ selects depth 8, minimum split 10, and minimum leaf 5 to prevent overfitting.

3) Radial Basis Function Support Vector Machine (RBF-SVM)

Nonlinear interactions among valley temperature, falling pressure, and humidity govern storm onset [2]. RBF-SVM tests whether a curved Gaussian kernel boundary improves storm separation and probability calibration over linear and single-tree models under RO2. Because radial basis distances depend on squared differences across all 22 predictors, each fold pairs z-score scaling with a balanced-weight RBF support vector classifier and tunes $C \in \{0.5, 1.0, 2.0\}$ via 5-fold time-series grid search ($C = 1.0$ selected). Internal 5-fold Platt sigmoid scaling converts margin distances into calibrated probabilities.

4) Random Forest

Single trees overfit yesterday's rain. Random Forest overcomes that single-variable trap for RO2 and RO3 by combining bootstrap tree averaging with random feature subsets at every node split [1, 18, 19]. Each tree grows on an independent bootstrap sample of the 8,759 unscaled training days (balanced class weights, seed 42) and checks only $\lfloor\sqrt{22}\rfloor = 4$ randomly drawn candidate predictors per split, so yesterday's rain (`precip_lag1`) is absent from roughly four out of five candidate draws and those splits must instead build rules from specific humidity, dew point, relative humidity, 3-day rain totals, or daily temperature range. Five-fold tuning selects 100 trees, depth 8, and minimum leaf size 5.

5) k-Nearest Neighbours (k-NN)

Weather analog forecasting predicts today's rain from the majority vote of the k closest historical weather days [2, 20]. An in-fold z-score scaled k-NN pipeline tests this approach under RO2 across $k \in \{5, 9, 15, 21\}$ and uniform versus distance weights, where 5-fold time-series validation selects $k = 5$ with uniform voting (also tested on the 7 PCA axes). Unweighted k-NN illustrates the accuracy paradox directly. Because non-extreme days outnumber storms 7-to-1 even inside the wet monsoon cluster, Euclidean neighbour voting in 22 dimensions favours the dry majority.

D. Validation, Runtimes, and Cost-Sensitive Threshold Design

Five-fold expanding-window time-series cross-validation inside the 1995–2018 training set (validation fold length $\approx 1,459$ days) tunes all five classifiers by out-of-fold F_1-score, and Table IV compares their fit and test speeds.

TABLE IV
TRAINING RUNTIMES, TEST INFERENCE SPEED, AND SCALING BEHAVIOUR ACROSS ALL EIGHT ALGORITHMS

Method / Classifier	Scaling Behaviour	Train Fit Time	Test Inference (2,192 d)
PCA (22 features)	Fast matrix decomposition	0.0028 s	Not applicable
FastICA (7 components)	Fast iterative projection	0.1383 s	Not applicable
K-Means (3 clusters)	Fast centroid distance updates	0.0804 s	Not applicable
Logistic Regression	Fast linear gradient optimisation	0.0533 s	0.0011 s
Decision Tree (CART)	Fast single-tree threshold sorting	0.0576 s	0.0006 s
RBF-Kernel SVM	Slow pairwise kernel optimization	10.4518 s	0.7772 s
Random Forest (100 trees)	**Moderate parallel tree building**	**1.2909 s**	**0.0290 s**
k-NN (5 neighbours)	Instant storage; slow test distance scan	0.0024 s	0.0702 s

Search grids appear in Table V, while Precision, Recall, harmonic F_1-score, and the Brier calibration score (BS) measure holdout skill across the 2,192 days of 2019–2024:

$$\begin{aligned} \text{Prec} &= \frac{\text{TP}}{\text{TP}+\text{FP}}, \quad \text{Rec} = \frac{\text{TP}}{\text{TP}+\text{FN}}, \\ F_1 &= \frac{2\,\text{TP}}{2\,\text{TP}+\text{FP}+\text{FN}}, \quad \text{BS} = \frac{1}{N}\sum_{i=1}^{N}(\hat{p}_i - y_i)^2 \end{aligned} \tag{1}$$

Block bootstrap intervals and McNemar tests check significance. Across the 5 expanding training folds, out-of-fold

storm probability predictions evaluate candidate cutoffs $\tau \in [0.05, 0.95]$ for RO3 under a 15-to-1 missed-flood cost function:

$$\text{Cost}(\tau) = 15 \times \text{FN}(\tau) + 1 \times \text{FP}(\tau) \tag{2}$$

which selects the operational cutoff $\tau^*_{\text{CV}} = 0.30$ strictly from training folds before holdout testing.

TABLE V
FIVE-FOLD TIME-SERIES CROSS-VALIDATION SEARCH GRIDS AND SELECTED MODEL CONFIGURATIONS

Classifier	Candidate Settings Tested	Selected Best Configuration	Train CV F1 (± SD)
Logistic Regression	Penalty C: 0.01, 0.1, 1.0, 10.0	C = 0.01, L2 penalty, balanced weights	0.3172 ± 0.0778
Decision Tree (CART)	Depth: 3, 4, 5, 6, 8; Leaf: 5, 10, 20	Max depth = 8, min leaf = 5, balanced	0.3383 ± 0.0566
RBF-Kernel SVM	Penalty C: 0.5, 1.0, 2.0; scale kernel	C = 1.0, RBF kernel, balanced, Platt	0.3476 ± 0.0514
Random Forest	**Trees: 100, 150; Depth: 4, 6, 8; Leaf: 5, 10**	**100 trees, max depth = 8, min leaf = 5**	**0.3780 ± 0.0429**
k-NN	Neighbours: 5, 9, 15, 21; uniform/distance	5 neighbours, uniform voting	0.2003 ± 0.0844

V. Results

A. Unsupervised Regime Findings (RO1)

1) Optimal Cluster Selection

Three clusters best fit Kathmandu's annual cycle across $K \in \{2,3,4,5,6\}$ on the 1995–2018 training set (Fig. 4). Within-cluster sum of squares (WCSS) drops sharply from 129.0×10^3 at $K = 2$ to an elbow at 98.1×10^3 for $K = 3$, while the full-sample Silhouette score peaks at 0.3119 ($K = 3$ vs. 0.2923 at $K = 2$) and the Davies–Bouldin index hits its minimum of 1.1327. Two clusters are too coarse: $K = 2$ merges cold winter days and hot pre-monsoon days into one dry group. Higher counts ($K = 4, 5$, or 6) steadily degrade separation (Silhouette drops to 0.2856, 0.2312, and 0.2017; Davies–Bouldin climbs to 1.4883 at $K = 5$) because extra centroids split dry transitional days rather than isolating storms. Three regimes match dry winter, hot pre-monsoon, and wet monsoon weather.

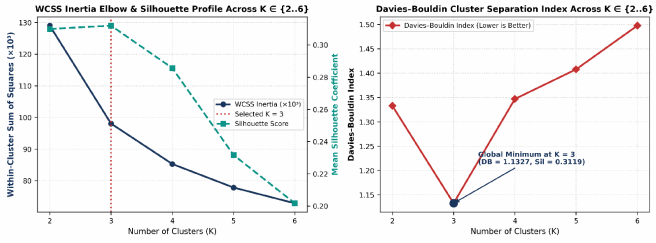


Fig. 4. Unsupervised K-Means cluster validation across 2 to 6 clusters on the 1995–2018 training set: inertia elbow and Silhouette peak at 3 clusters (left), and Davies–Bouldin minimum at 3 clusters (right).

2) PCA and FastICA Component Structure

Two principal axes capture 66.46% of total variance across the 22 training predictors (Fig. 5, left; Fig. 6, left). PC1 explains 42.29% of training variance ($\lambda_1 = 9.30$) and represents monsoon moisture build-up with positive loadings on specific humidity (`QV2M`, +0.313), dew point (`T2MDEW`, +0.307), minimum temperature (`T2M_MIN`, +0.282), relative humidity (`RH2M`, +0.265), and 7-day rain (+0.261) against negative loadings on surface pressure (`PS`, −0.245) and daily temperature range (−0.233). PC2 adds 24.17% ($\lambda_2 = 5.32$) from pre-monsoon wind speed (+0.360), maximum temperature (+0.357), and solar radiation (+0.332).

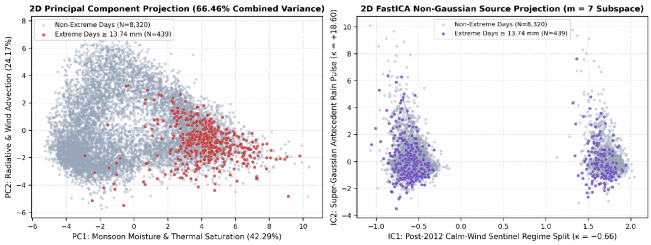


Fig. 5. Two-dimensional projections of the 22-feature training set by extreme rainfall status: PCA projection with 66.46% variance (left) and FastICA independent source projection (right).

Rain memory drives PC3 ($\lambda_3 = 2.01$). Its top loadings fall on 3-day and 2-day rain lags (+0.440 and +0.407; 9.15% variance), while seven PCA axes retain 91.46% of total variance (95.11% at nine axes; Fig. 6, left) and FastICA separates the post-2012 calm-wind shift on IC1 ($\kappa = -0.66$) from sharp multi-day storm pulses on IC2 ($\kappa = +18.60$; Fig. 5, right).

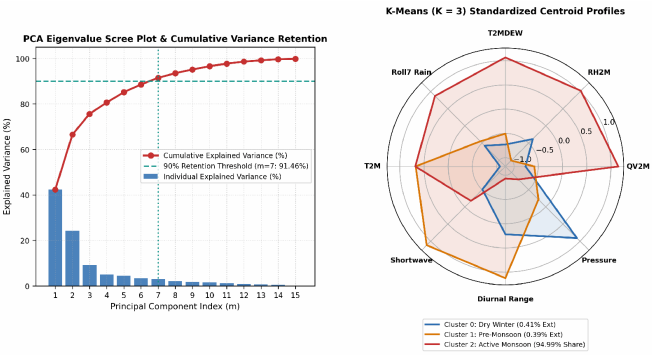


Fig. 6. PCA explained variance scree plot with 91.46% retention at 7 components (left), paired with the K-Means (3 clusters) radar profile across eight core weather variables (right).

3) Physical Profiles of the Three Weather Regimes

Average weather conditions and storm counts across the three K-Means clusters appear in Table VI and Fig. 6 (right):

- **Cluster 0 (Dry Winter / Post-Monsoon, 3,395 days, 38.76%):** Cool, high-pressure dry air (13.90°C, QV2M = 5.57 g/kg, PS = 87.63 kPa, 7-day rain = 2.28 mm). Only 14 storms fall here (0.41% rate; 3.19% of training storms).
- **Cluster 1 (Hot Pre-Monsoon Transitional, 2,067 days, 23.60%):** Warm, sunny dry weather (22.56°C, RH2M = 34.02%, daily temperature range = 13.02°C, solar radiation = 22.82 MJ/m^2/d) with just 8 storms (0.39% rate; 1.82% of storms).
- **Cluster 2 (Active Saturated Monsoon, 3,297 days, 37.64%):** Warm, humid low-pressure monsoon air (QV2M = 15.85 g/kg, RH2M = 80.92%, T2MDEW = 18.76°C, daily range = 6.80°C, PS = 86.95 kPa, 7-day rain = 42.26 mm). This single cluster holds **94.99% of all training storms** (417 of 439), yet 2,880 of its days (87.35%) are ordinary wet days that still require supervised classification.

TABLE VI
UNSUPERVISED K-MEANS (3 CLUSTERS) WEATHER REGIME PROFILES AND EXTREME STORM CONCENTRATION (1995–2018)

Weather Variable / Metric	Cluster 0: Dry Winter	Cluster 1: Pre-Monsoon	Cluster 2: Active Monsoon	All Training Days
Total Training Days (Share)	3,395 (38.76%)	2,067 (23.60%)	**3,297 (37.64%)**	8,759 (100.0%)
Specific Humidity QV2M (g/kg)	5.57	6.54	**15.85**	9.67
Relative Humidity RH2M (%)	48.56	34.02	**80.92**	57.31
Dew Point T2MDEW (°C)	1.97	4.07	**18.76**	8.78
Mean Temperature T2M (°C)	13.90	22.56	**22.60**	19.22
Daily Temp. Range DTR (°C)	10.29	13.02	**6.80**	9.62
Surface Pressure PS (kPa)	87.63	87.18	**86.95**	87.27
Solar Radiation (MJ/m^2/d)	15.20	22.82	**16.77**	17.60
7-Day Past Rainfall (mm)	2.28	5.83	**42.26**	18.17
Extreme Rain Days (Rate)	14 (0.41%)	8 (0.39%)	**417 (12.65%)**	439 (5.01%)
Share of All 439 Extremes	3.19%	1.82%	**94.99%**	100.00%

B. Supervised Classifier Comparison (RO2)

1) Holdout Test Performance Across All Five Models

Holdout testing covers 2019–2024. Across those 2,192 unseen days (164 extreme rain days, or 7.48% storm prevalence), Table VII, Fig. 7, Fig. 8, and Fig. 9 compare all five classifiers at the default $\tau = 0.50$ cutoff through their test metrics, ROC curves, Precision-Recall curves, and confusion matrices.

TABLE VII
HOLDOUT TEST PERFORMANCE ACROSS ALL FIVE SUPERVISED CLASSIFIERS (2019–2024, 2,192 DAYS, 164 EXTREMES, CUTOFF 0.50)

Evaluation Metric	Logistic Reg.	Decision Tree	RBF-SVM	Random Forest	k-NN (k=5)
Overall Accuracy (%)	75.87	81.07	79.06	82.71	**93.16**
Precision (%)	23.20	26.45	24.70	28.96	**58.75**
Recall / Storm Detection (%)	**96.34**	85.98	87.80	**90.24**	28.66
F1-Score	0.3740	0.4046	0.3855	**0.4385**	0.3852
95% Block-Bootstrap CI (F1)	[31.97, 42.63]	[34.75, 45.44]	[33.07, 43.70]	**[38.17, 49.14]**	[30.77, 44.90]
Specificity (%)	74.21	80.67	78.35	82.10	**98.37**
ROC-AUC	**0.9411**	0.8718	0.9011	**0.9349**	0.8219
Precision-Recall AUC	0.5478	0.4068	0.3676	**0.5517**	0.3547
Brier Calibration Score	0.1621	0.1418	**0.0553**	0.1154	0.0560
True Positives (Caught Storms)	**158**	141	144	**148**	47
False Negatives (Missed Storms)	**6**	23	20	**16**	117
False Positives (False Alarms)	523	392	439	363	**33**
True Negatives (Correct Dry)	1,505	1,636	1,589	1,665	**1,995**

2) Random Forest: Best Overall Operational Balance

Random Forest ranks first in balanced storm skill. Across the 2019–2024 holdout test set (Table VII), the 100-tree ensemble records the top F_1-score at 0.4385 (7-day block-bootstrap 95% CI: $[38.17\%, 49.14\%]$) and top Precision-Recall AUC at 0.5517 (over 7.3 times the 0.0748 baseline) and catches 148 of 164 storms (90.24% recall; 16 missed) at 82.71% accuracy, 0.9349 ROC-AUC, 363 false alarms, and a 0.0290 s test runtime.

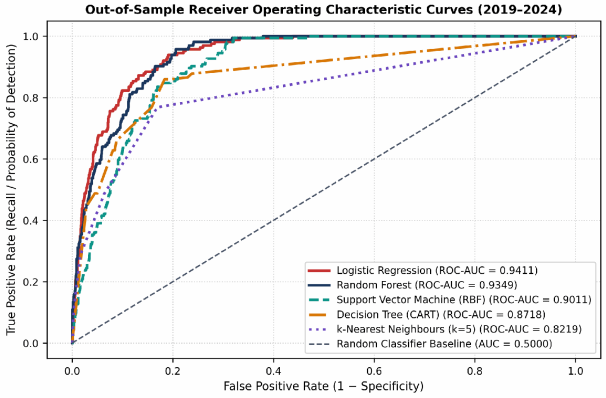


Fig. 7. Receiver Operating Characteristic (ROC) curves across all five classifiers on the 2019–2024 holdout test set (2,192 days).

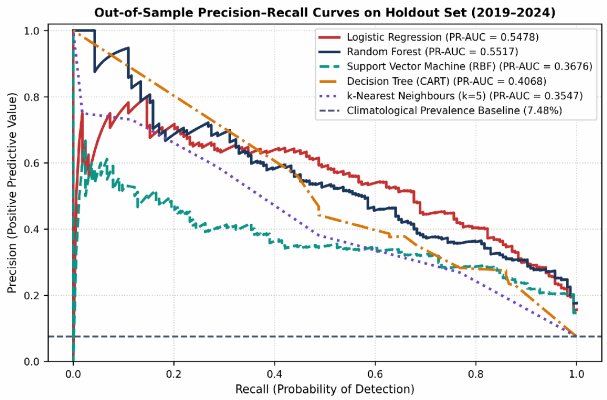


Fig. 8. Precision-Recall curves on the 2019–2024 holdout test set against the 7.48% storm prevalence baseline, where Random Forest (0.5517) and Logistic Regression (0.5478) achieve over a seven-fold gain above random chance.

3) Logistic Regression: Maximum Storm Sensitivity

Balanced Logistic Regression catches more default-cutoff storms than any other model. Across the 6-year holdout window, it detects 158 of 164 storms (96.34% recall; only 6 missed) and posts the highest ROC-AUC (0.9411) alongside a 0.5478 Precision-Recall AUC, though its flat linear boundary triggers 523 false alarms on warm, humid monsoon break days (23.20% precision; $F_1 = 0.3740$).

4) Decision Tree (CART): Rule Baseline and Single-Variable Reliance

A single tuned CART tree ranks second in F_1-score at 0.4046. On the holdout set, CART reaches 81.07% accuracy, 85.98% recall (141 storms caught, 23 missed), 26.45% precision (392 false alarms), 0.8718 ROC-AUC, and 0.4068 PR-AUC; Random Forest's 100-tree ensemble beats this single tree by catching 7 more storms and cuts 29 false alarms and lifting PR-AUC by 14.49 percentage points.

5) RBF-SVM: Strong Calibration but Lower Precision-Recall Skill

Platt-scaled RBF-SVM posts the lowest Brier calibration error (0.0553) of all five models. It catches 144 of 164 holdout storms (87.80% recall, 439 false alarms, $F_1 = 0.3855$, ROC-AUC $= 0.9011$), but high-probability false alarms on humid non-storm days pull its Precision-Recall AUC down to 0.3676 and kernel fitting takes 10.45 s.

6) k-Nearest Neighbours: Demonstration of the Accuracy Paradox

Overall accuracy misleads on skewed flood data. Unweighted k-NN ($k = 5$) tops Table VII in overall accuracy (93.16%) and precision (58.75%), yet it misses 117 of 164 holdout storms (28.66% recall; only 47 caught in Fig. 9) because 92.52% of test days are non-extreme.

7) Feature Importance and Physical Drivers

Feature rankings in Table VIII and Fig. 10 show why Random Forest beats a single CART tree. Because CART checks all 22 columns greedily at every split, it puts 73.35% of its Gini weight on yesterday's rain (`precip_lag1`) and 0.00% on specific humidity (`QV2M`) and 7-day rain (`precip_roll7`). Random Forest draws only 4 random candidate columns per split. Weight spreads across multi-day rain memory (`precip_lag1` 20.83%, `precip_roll3` 11.55%,

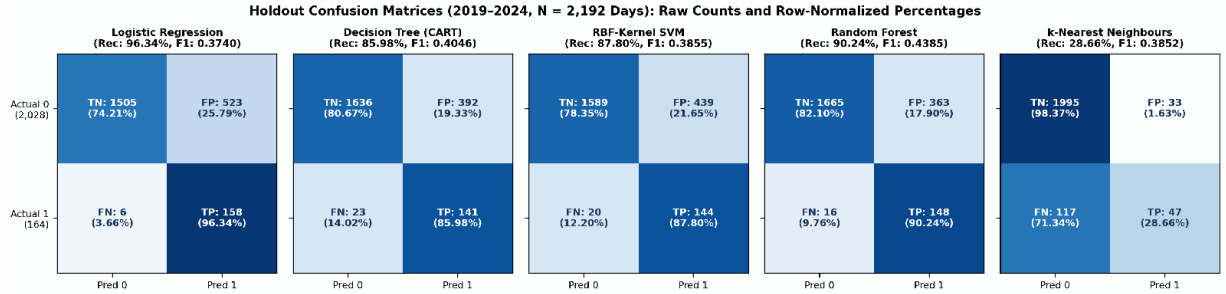


Fig. 9. Holdout confusion matrices across all five classifiers (2019–2024, 2,192 days: 2,028 non-extreme, 164 extreme) with raw counts and row percentages.

`precip_roll7` 5.24%), water vapour (`QV2M` 10.37%, `T2MDEW` 9.36%, `RH2M` 5.43%, `dew_point_depression` 5.12%), and cloud cooling (`temp_diurnal_range` 7.00%, `ALLSKY_SW` 4.41%), while Logistic Regression weights confirm that yesterday's rain ($\beta = +0.4969$) and relative humidity ($\beta = +0.4679$) raise storm risk and sunshine ($\beta = -0.5355$) and wide temperature ranges ($\beta = -0.4886$) favour dry skies.

TABLE VIII
TOP-10 WEATHER PREDICTOR IMPORTANCE RANKINGS ACROSS MODELS

Rank	Predictor Variable	Random Forest	Decision Tree	Logistic Reg. Weight	Permutation Drop
1	precip_lag1 (Yesterday rain)	**0.2083**	**0.7335**	**+0.4969**	**0.0721**
2	precip_roll3 (3-day rain)	0.1155	0.0109	+0.1092	0.0105
3	QV2M (Specific humidity)	0.1037	0.0000	+0.2888	0.0034
4	T2MDEW (Dew point)	0.0936	0.0037	+0.3057	0.0012
5	temp_diurnal_range	0.0700	0.0461	-0.4886	0.0099
6	RH2M (Relative humidity)	0.0543	0.0122	+0.4679	0.0070
7	precip_roll7 (7-day rain)	0.0524	0.0000	+0.0756	0.0018
8	dew_point_depression	0.0512	0.0086	-0.3168	0.0039
9	ALLSKY_SW (Solar radiation)	0.0441	0.0322	**-0.5355**	0.0206
10	T2M_MIN (Min temperature)	0.0349	0.0097	+0.2000	0.0005

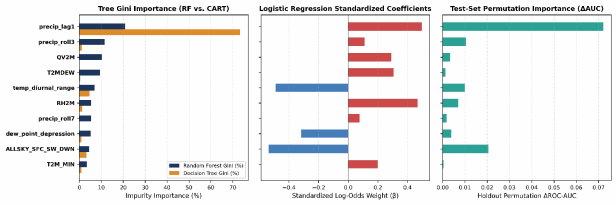


Fig. 10. Comparison of top-10 weather predictors: Random Forest versus Decision Tree Gini importance (left), Logistic Regression standardized weights (centre), and holdout test permutation importance (right).

8) *Probability Calibration and Subspace Ablation*

Reliability curves and Brier scores in Fig. 11 show how class weighting shifts predicted probabilities. Platt-scaled RBF-SVM (0.0553) and unweighted k-NN (0.0560) post the smallest raw Brier errors because their predictions cluster near the 7.48% base rate, whereas balanced class weights in Random Forest (0.1154), CART (0.1418), and Logistic Regression (0.1621) lift storm probabilities upward to prevent missed floods.

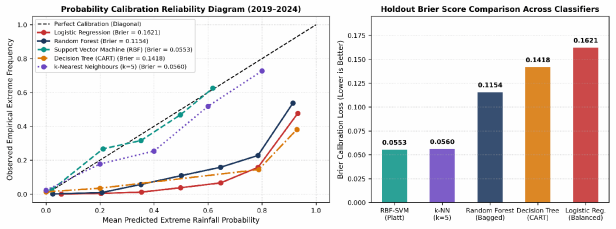


Fig. 11. Probability calibration curves across all five classifiers on the 2019–2024 holdout test set (left) and Brier score comparison between balanced class weighting and Platt scaling (right).

Paired McNemar tests confirm these gaps (Table IX, Part A). Across the 164 holdout storms, Random Forest catches 101 events missed by k-NN while k-NN catches zero events missed by Random Forest ($\chi^2 = 99.01, p < 10^{-30}$), and replacing the 22 physical predictors with the 7 PCA components (Part B) lowers recall from 96.34% to 93.29% for Logistic Regression (F_1: 0.3740 to 0.3562) and from 28.66% to 24.39% for k-NN (F_1: 0.3852 to 0.3265).

TABLE IX
PAIRED MCNEMAR SIGNIFICANCE TESTS AND 7-COMPONENT PCA VERSUS 22-FEATURE ABLATION (2019–2024 TEST SET)

Part A: Paired Model Comparison	**All Days Disagreed**	**Chi-Square (p-value)**	**Storms Disagreed**	**Storm Chi-Square (p)**
Random Forest vs. k-NN	106 vs. 335	117.88 ($< 10^{-28}$)	**101 vs. 0**	**99.01** ($< 10^{-30}$)
Logistic Regression vs. k-NN	112 vs. 491	236.96 ($< 10^{-56}$)	**111 vs. 0**	**109.01** ($< 10^{-33}$)
Random Forest vs. Decision Tree	99 vs. 63	7.56 (0.0058)	13 vs. 6	1.89 (0.170)
Random Forest vs. RBF-SVM	135 vs. 55	32.85 ($< 10^{-8}$)	13 vs. 9	0.41 (0.520)
Logistic Reg. vs. Random Forest	22 vs. 172	114.44 ($< 10^{-29}$)	11 vs. 1	6.75 (0.0063)
Part B: PCA Subspace Ablation	**Accuracy (%)**	**Storm Recall (%)**	**F1-Score**	**ROC / PR-AUC**
Logistic Reg. (Full 22 Features)	**75.87**	**96.34**	**0.3740**	**0.9411 / 0.5478**
Logistic Reg. (7 PCA Modes, 91.5%)	74.77	93.29	0.3562	0.9287 / 0.5240
k-NN (Full 22 Features)	**93.16**	**28.66**	**0.3852**	**0.8219 / 0.3547**
k-NN (7 PCA Modes, 91.5%)	92.47	24.39	0.3265	0.8159 / 0.3053

C. *Operational and Flood Case Study Results (RO3)*

1) *Cost-Sensitive Threshold Optimisation*

A standard 0.50 cutoff penalises missed floods and false alarms equally. Under the 15-to-1 operational cost rule (Cost = 15 FN + 1 FP; Table X), the default $\tau = 0.50$ Random Forest threshold misses 16 holdout storms and triggers 363 false alarms for a total penalty of 603.

TABLE X
RANDOM FOREST COST-SENSITIVE THRESHOLD SWEEP UNDER THE 15-TO-1 MISSED-STORM PENALTY (2019–2024 TEST SET)

Probability Cutoff & Rule	**Acc (%)**	**Prec (%)**	**Recall (%)**	**F1**	**Caught / Missed**	**False Alarms**	**15:1 Cost**
0.10 (High Standby Watch)	61.59	16.30	100.00	0.2803	164 / 0	842	842
0.20 (Low Warning Cutoff)	70.44	20.12	99.39	0.3347	163 / 1	647	662
0.30 (Prospective Training CV)	**76.82**	**24.17**	**98.17**	**0.3880**	**161 / 3**	**505**	**550 (OOF)**
0.35 (Intermediate Cutoff)	78.38	25.16	95.73	0.3985	157 / 7	467	572
0.40 (Retrospective Test Min)	80.20	26.88	95.73	0.4198	157 / 7	427	532 (Test)
0.50 (Standard Default Cutoff)	82.71	28.96	90.24	0.4385	148 / 16	363	603
0.60 (Moderate Confidence)	84.72	31.13	85.98	0.4571	141 / 23	312	657
0.70 (High-Priority Red Alert)	87.96	36.19	79.88	**0.4981**	131 / 33	231	726

In Fig. 12, the training-tuned threshold $\tau^*_{CV} = 0.30$ lifts holdout recall from 90.24% to **98.17%** (161 of 164 storms caught), cuts missed floods by 81.25% (from 16 down to 3), and drops total disaster cost from 603 to 550. The post-hoc test minimum sits at $\tau^* = 0.40$ (cost = 532).

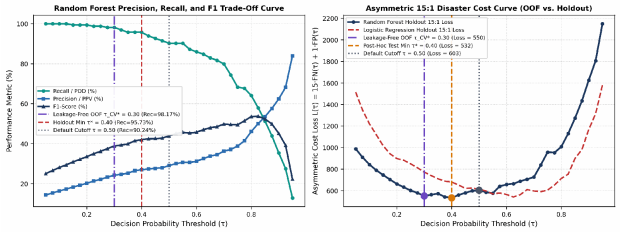


Fig. 12. Random Forest precision, recall, and F1 trade-off curves across probability cutoffs (left) and the 15-to-1 disaster cost curve for the training-derived 0.30 threshold versus the 0.40 test minimum and 0.50 default cutoff (right).

2) Retrospective Validation on the September 2024 Flood

Day-1 storm onset is the toughest test for any flood model. Four days of light showers (0.39 to 1.14 mm/day on 20–23 September 2024; Random Forest $\hat{p} \leq 0.1831$) preceded five days of extreme rain at 47.54, 55.89, 16.56, 76.28, and 93.26 mm/day across 24–28 September (289.53 mm total; Fig. 13) [6, 7]. Yesterday's rain stood at just 0.58 mm on 24 September, yet same-day Logistic Regression caught the sharp rise in relative humidity (90.47%) and narrow daily temperature range (3.66°C) at $\hat{p} = 0.7154$; from 25 to 28 September, 1-day and 3-day rain lags pushed both Random Forest ($\hat{p} = 0.6881$ to 0.7189) and Logistic Regression ($\hat{p} = 0.8674$ to 0.9997) well above the 0.30 and 0.50 alert lines.

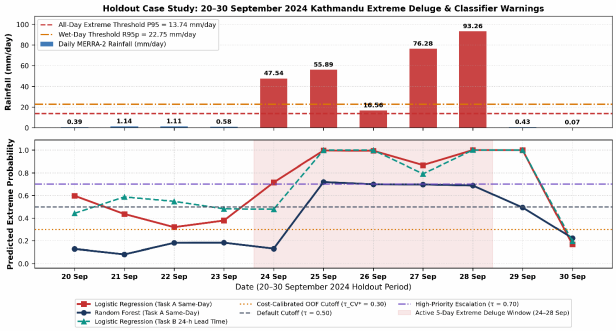


Fig. 13. Retrospective timeline of the 20–30 September 2024 Kathmandu flood disaster: daily rainfall against the 13.74 mm/day and 22.75 mm/day thresholds (top), and predicted storm probabilities from Random Forest and Logistic Regression (bottom).

3) 24-Hour Advance Forecasting and Wet-Day Benchmarks

Lead-time and wet-day sensitivity tests appear in Table XI and Fig. 14. Under strict 24-hour advance forecasting (Task B, where all 15 atmospheric predictors shift back to day $t-1$), Random Forest leads in F_1-score (0.4097) and ROC-AUC (0.9135) and catches 144 of 164 holdout storms a full day ahead (87.80% recall, just 2.44 points below same-day Task A), while Logistic Regression flags 152 of 164 storms 24 hours ahead (92.68% recall, PR-AUC = 0.4526). Unweighted k-NN falls to 17.68% recall on Task B (135 storms missed). For severe wet-day deluges ≥ 22.75 mm/day (Task C, 64 holdout events), Logistic Regression detects 62 of 64 deluges (96.88% recall, ROC-AUC = 0.9461) and Random Forest detects 57 of 64 (89.06% recall, ROC-AUC = 0.9276).

TABLE XI
COMPARISON OF SAME-DAY CLASSIFICATION (TASK A), STRICT 24-HOUR ADVANCE FORECASTING (TASK B), AND WET-DAY DELUGE DETECTION (TASK C)

Forecasting Task & Model	Acc (%)	Prec (%)	Recall (%)	F1	ROC-AUC	PR-AUC	TP / FN / FP
Task A (Same-Day, 13.74 mm) – RF	82.71	28.96	90.24	**0.4385**	0.9349	**0.5517**	148 / 16 / 363
Task A (Same-Day, 13.74 mm) – LR	75.87	23.20	**96.34**	0.3740	**0.9411**	0.5478	158 / 6 / 523
Task B (24-Hr Lead, 13.74 mm) – RF	81.07	26.72	87.80	**0.4097**	**0.9135**	0.4194	144 / 20 / 395
Task B (24-Hr Lead, 13.74 mm) – LR	72.31	20.35	**92.68**	0.3337	0.9088	**0.4526**	152 / 12 / 595
Task B (24-Hr Lead, 13.74 mm) – k-NN	**92.29**	**46.03**	17.68	0.2555	0.7812	0.2509	29 / 135 / 34
Task C (Wet-Day, 22.75 mm) – RF	83.85	14.11	89.06	**0.2436**	0.9276	0.2851	57 / 7 / 347
Task C (Wet-Day, 22.75 mm) – LR	78.01	11.44	**96.88**	0.2046	**0.9461**	**0.3985**	62 / 2 / 480

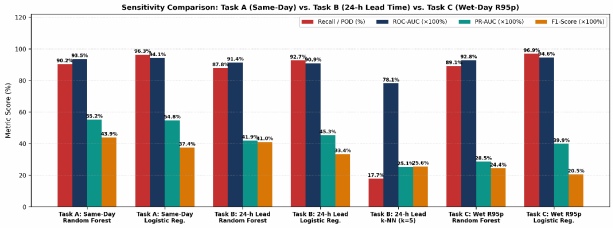


Fig. 14. Holdout performance comparison across Task A (same-day classification at 13.74 mm/day), Task B (strict 24-hour advance forecasting), and Task C (severe wet-day classification at 22.75 mm/day).

VI. Discussion and Conclusions

A. Subspace Compression, the Accuracy Paradox, and Ensemble Complementarity

1) Effect of PCA Compression on Storm Signals

Orthogonal variance compression hurts rare-storm recall. Even though 7 PCA axes retain 91.46% of training variance across the 22 predictors, training on those 7 axes lowers holdout recall by 3.05 percentage points for Logistic Regression and 4.27 points for k-NN (Table IX). Because PCA orders axes by full-year variance, its leading components favour smooth seasonal temperature and humidity waves while smoothing out abrupt 1-day jumps in yesterday's rain (`precip_lag1`) and cloud-cooled daily temperature range (`temp_diurnal_range`) [16, 17]. Full 22-feature tables keep those short-term storm triggers intact.

2) Why Unweighted k-NN Suffers the Accuracy Paradox

Local neighbour voting fails because non-extreme wet days outnumber cloudbursts inside the monsoon season itself. Unweighted k-NN reaches 93.16% overall accuracy on 2019–2024 yet misses 71.34% of same-day storms (Table VII) and 82.32% of 24-hour storms (Table XI), which contradicts earlier urban flood claims [2]. Inside Cluster 2 (Active Monsoon; Table VI), where 94.99% of training storms occur, 87.35% of days are still ordinary wet days, so the 5 closest historical neighbours of a developing storm almost always vote for non-extreme rain.

3) Complementary Strengths of Random Forest and Logistic Regression

Random Forest and balanced Logistic Regression cover distinct operational needs. A single CART tree puts 73.35% of its split weight on `precip_lag1` (Table VIII) and lags on Day 1 of a storm. By restricting each split to 4 random candidate columns, Random Forest spreads weight across 3-day rain totals, specific humidity, dew point, and daily temperature range and cuts 160 false alarms compared with Logistic Regression, whereas balanced Logistic Regression acts as an immediate moisture-surge screen that caught the sudden 24 September 2024 onset ($\hat{p} = 0.7154$) before both models locked onto the 25–28 September peak.

B. Three-Tier Operational Flood Warning Protocol

To balance early storm detection against false-alarm fatigue at Tribhuvan International Airport and municipal drainage offices, the prototype dashboard (Appendix D) uses a three-step alert ladder:

- **Tier 1 – Yellow Watch (Internal Forecaster Screening):** Starts 24 hours ahead when Task B Logistic Regression hits $\hat{p} \geq 0.50$ (92.68% recall) or same day when

Task A Random Forest reaches the 15:1 cutoff $\tau^*_{CV} = 0.30$ (98.17% recall; 161 of 164 storms). Forecasters monitor radar and gauges internally.

- **Tier 2 – Orange Standby (Municipal Preparedness):** Starts when same-day Random Forest hits $\hat{p} \geq 0.40$ or Logistic Regression hits $\hat{p} \geq 0.70$ (95.73% recall, which catches the 24 September 2024 onset) and puts city drainage crews and airport staff on standby.
- **Tier 3 – Red Escalation (High-Priority Flood Warning):** Starts when Random Forest exceeds $\hat{p} \geq 0.68$ and Logistic Regression exceeds $\hat{p} \geq 0.85$ together (25–28 September 2024 peak) alongside live rain-gauge confirmation.

C. Ethical, Legal, and Methodological Limitations

Public-interest duties under the British Computer Society (BCS) Code of Conduct govern flood warning deployment. Low-income riverside settlements (*Sukumbasi* communities) along the Bagmati and Bishnumati corridors face the highest flood exposure in Kathmandu [6, 7]: an accuracy-tuned k-NN model that misses 71% of floods would leave those households unwarned, while unverified public alarms would trigger panic. Nepal's *Disaster Risk Reduction and Management Act, 2074 (2017)* assigns sole legal authority for public flood warnings to the Department of Hydrology and Meteorology (DHM) and the National Disaster Risk Reduction and Management Authority [6], so this dashboard serves professional meteorologists only. Three technical limits also remain: one $0.5^\circ \times 0.625^\circ$ ($\sim$ 50 km) MERRA-2 grid box smooths local peaks (93.26 mm/day on 28 September 2024 vs. $>$ 240 mm at point gauges [6, 7]), daily totals mask 1-hour cloudburst rates, and surface variables omit upper-air wind divergence.

D. Conclusions

1) Achievement of RO1: Data and Unsupervised Regime Analysis

The 30-year MERRA-2 pipeline (10,951 post-lag days; 22 predictors) fulfilled RO1. It isolated post-2012 calm-wind codes, restricted rolling rain sums to past days ($t-1$ to $t-7$), and split the record at 31 December 2018 (8,759 train and 2,192 holdout days) around training-only thresholds (13.74 and 22.75 mm/day), while 7 PCA axes kept 91.46% of variance, FastICA separated sharp rain pulses, and K-Means ($K = 3$, Silhouette $= 0.3119$) placed 94.99% (417 of 439) of training storms inside the Active Monsoon cluster.

2) Achievement of RO2: Comparative Classifier Evaluation

Five classifiers competed under 5-fold time-series cross-validation for RO2. Random Forest achieved the best balanced holdout skill ($F_1 = 0.4385$, PR-AUC $= 0.5517$, ROC-AUC $= 0.9349$, 90.24% recall; 148 of 164 storms) by breaking CART's 73.35% reliance on yesterday's rain, whereas balanced Logistic Regression reached 96.34% recall (158 of 164 storms; ROC-AUC $= 0.9411$), Platt-scaled RBF-SVM posted the lowest Brier error (0.0553), and unweighted k-NN exposed the accuracy paradox (93.16% accuracy but 28.66% recall).

3) Achievement of RO3: Operational and Cost-Sensitive Validation

Cost-sensitive threshold calibration satisfied RO3. At the training-tuned Random Forest cutoff $\tau^*_{CV} = 0.30$ under a 15-to-1 missed-flood penalty, holdout recall rose from 90.24% to 98.17% (161 of 164 storms caught, missed floods cut from 16 to 3, and cost reduced from 603 to 550), while high recall held at a 24-hour lead time (Task B: Random Forest 87.80%, Logistic Regression 92.68%), on wet-day deluges $\geq$ 22.75 mm/day (Task C: Logistic Regression 96.88%), and across the 24–28 September 2024 Kathmandu flood.

4) Directions for Future Work

Sub-daily cloudbursts require finer spatial resolution. Future work should pair MERRA-2 predictors with hourly DHM rain gauges, valley weather radar, and hydraulic river-routing models for the Bagmati and Bishnumati basins.

References

[1] S. Poudyal, S. Katwal, and R. Shakya, "Rainfall prediction in Kathmandu City using machine learning and deep learning techniques," *Int. J. Eng. Technol. Infrastruct. Dev. (InJET-InDev)*, vol. 2, no. 2, pp. 18–25, May 2026, doi: 10.3126/injet-indev.v2i2.95697.

[2] T. Nurmayati, "Performance comparison of Random Forest, Support Vector Machine, and K-Nearest Neighbors algorithms in ten-minute rainfall prediction for urban flood mitigation in South Tangerang," *J. Appl. Inform. Comput. (JAIC)*, vol. 10, no. 4, pp. 3911–3919, Aug. 2026, doi: 10.30871/jaic.v10i4.10809.

[3] Z.-S. Liu, M. Boy, and R. Makkonen, "TailWeather: From tail to extremes, a global climatological dataset for machine-learning weather forecasting," *arXiv preprint* arXiv:2609.12496v1, Sep. 2026. [Online]. Available: https://arxiv.org/abs/2609.12496v1

[4] D. Chaulagain, R. L. Ray, A. O. Yakub, *et al.*, "Extreme precipitation dynamics and El Niño-Southern Oscillation influences in Kathmandu Valley, Nepal," *Water*, vol. 17, no. 9, p. 1397, May 2025, doi: 10.3390/w17091397.

[5] K. Khanal, B. Baniya, M. Sigdel, *et al.*, "Spatial and temporal variation of precipitation extreme in the Kathmandu Valley during last three decades," *J. Hydrol. Meteorol.*, vol. 12, no. 1, pp. 34–46, Dec. 2024, doi: 10.3126/jhm.v12i1.72653.

[6] Department of Hydrology and Meteorology (DHM), *Situational Report on Extreme Precipitation and Flooding Event of 27–29 September 2024*, Government of Nepal, Kathmandu, Nepal, Oct. 2024. [Online]. Available: https://www.dhm.gov.np/

[7] International Centre for Integrated Mountain Development (ICIMOD), "Death toll climbs as torrential rains pound Nepal," *ICIMOD Situational Analysis*, Kathmandu, Nepal, Sep. 2024. [Online]. Available: https://www.icimod.org/death-toll-climbs-as-torrential-rains-pound-nepal/

[8] S. Phuyal, "Precipitation in Kathmandu Valley: A spatial-temporal study," *Int. J. Environ. Clim. Change*, vol. 14, no. 9, pp. 266–278, Sep. 2024, doi: 10.9734/ijecc/2024/v14i94410.

[9] O. A. Wani, S. S. Mahdi, M. Yeasin, *et al.*, "Predicting rainfall using machine learning, deep learning, and time series models across an altitudinal gradient in the North-Western Himalayas," *Sci. Rep.*, vol. 14, Art. no. 27876, Nov. 2024, doi: 10.1038/s41598-024-77687-x.

[10] M. Pujara and N. Paudel, "Rainfall prediction using Long Short-Term Memory and Gated Recurrent Unit with various meteorological parameters," *Nepalese J. Stat.*, vol. 8, no. 1, pp. 47–60, 2024, doi: 10.3126/njs.v8i1.73165.

[11] B. Bashyal, D. Bhusal, K. Singh, and R. Koju, "Rainfall prediction using wavelet transform and transformers," *KEC J. Sci. Eng.*, vol. 9, no. 1, pp. 144–153, 2025, doi: 10.3126/kjse.v9i1.78379.

[12] B. Paneru and B. Paneru, "Real-time rainfall prediction in Kathmandu, Kapan area using sensor data with machine learning and linear regression," *J. Soft Comput. Paradigm*, vol. 5, no. 3, pp. 266–286, Sep. 2023, doi: 10.36548/jscp.2023.3.004.

[13] S. Sayago *et al.*, "Comparative evaluation of NASA POWER, ERA5, and observational data for accuracy and reliability," *Theor. Appl. Climatol.*, vol. 156, Jun. 2025, doi: 10.1007/s00704-025-05512-w.

[14] K. Bi, L. Xie, H. Zhang, X. Chen, X. Gu, and Q. Tian, "Accurate medium-range global weather forecasting with 3D neural networks," *Nature*, vol. 619, no. 7970, pp. 533–538, Jul. 2023, doi: 10.1038/s41586-023-06185-3.

[15] R. Lam, A. Sanchez-Gonzalez, M. Willson, *et al.*, "Learning skillful medium-range global weather forecasting," *Science*, vol. 382, no. 6677, pp. 1416–1421, Dec. 2023, doi: 10.1126/science.adi2336.

[16] T. Talan, "Machine learning-based rainfall prediction across temporal scales: Model benchmarking and explainability analysis," *Stoch. Environ. Res. Risk Assess.*, vol. 40, Art. no. 100, 2026, doi: 10.1007/s00477-026-03245-8.

[17] Z. Zhang, Y. Liu, and H. Chen, "Identification and temporal distribution of typical rainfall types using PCA dimension reduction and K-means++ clustering," *Hydrology*, vol. 12, no. 4, p. 88, Apr. 2025, doi: 10.3390/hydrology12040088.

[18] Y. Wu, Z. Zhang, X. Qi, W. Hu, and S. Si, "Prediction of flood sensitivity based on Logistic Regression, eXtreme Gradient Boosting, and Random Forest modeling methods," *Water Sci. Technol.*, vol. 89, no. 10, pp. 2605–2624, May 2024, doi: 10.2166/wst.2024.146.

[19] X. Liu *et al.*, "Influencing factors and risk assessment of precipitation-induced flooding in Zhengzhou, China, based on Random Forest and XGBoost algorithms," *Int. J. Environ. Res. Public Health*, vol. 19, no. 24, p. 16544, Dec. 2022, doi: 10.3390/ijerph192416544.

[20] S. M. Toufique, S. U. Bhuiyan, A. Lateef, A. Zaman, J. Bin Islam, and D. Z. Karim, "Implementing machine learning techniques to forecast floods in Bangladesh," in *Proc. 2024 Int. Conf. Electr., Comput. Energy Technol. (ICECET)*, IEEE, Jul. 2024, pp. 1–6, doi: 10.1109/ICECET61485.2024.10698703.

APPENDIX

GitHub Link: https://github.com/gurungbishow/STW7072CEM-Machine-Learning.git
Video Link: https://youtu.be/1XqPLvwBD_4

A. Data Preprocessing and Feature Engineering Implementation

data_processing.py > ...

```python
import os
import json
import numpy as np
import pandas as pd

FEATURE_COLS_TASK_A = [
    "T2M", "T2M_MAX", "T2M_MIN", "temp_diurnal_range",
    "T2MDEW", "dew_point_depression", "RH2M", "QV2M",
    "PS", "pressure_tendency", "WS10M", "WS10M_MAX",
    "ALLSKY_SFC_SW_DWN", "TS", "wind_calm_flag",
    "precip_lag1", "precip_lag2", "precip_lag3",
    "precip_roll3", "precip_roll7",
    "sin_doy", "cos_doy",
]

FEATURE_COLS_TASK_B = [
    "T2M_lag1", "T2M_MAX_lag1", "T2M_MIN_lag1", "temp_diurnal_range_lag1",
    "T2MDEW_lag1", "dew_point_depression_lag1", "RH2M_lag1", "QV2M_lag1",
    "PS_lag1", "pressure_tendency_lag1", "WS10M_lag1", "WS10M_MAX_lag1",
    "ALLSKY_SFC_SW_DWN_lag1", "TS_lag1", "wind_calm_flag_lag1",
    "precip_lag1", "precip_lag2", "precip_lag3",
    "precip_roll3", "precip_roll7",
    "sin_doy", "cos_doy",
]


def resolve_dataset_path(csv_path=None):
    """Resolves the Kathmandu NASA POWER MERRA-2 CSV path across working
    directories."""
    base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    candidates = []
    if csv_path:
        candidates.append(csv_path)
    candidates.extend([
        os.path.join(base_dir, "data", "kathmandu_merra2_1995_2024.csv"),
        os.path.join(base_dir, "data",
        "kathmandu_airport_daily_meteorology_1995_2024.csv"),
        "data/kathmandu_merra2_1995_2024.csv",
        "data/kathmandu_airport_daily_meteorology_1995_2024.csv",
        "../data/kathmandu_merra2_1995_2024.csv",
        "../data/kathmandu_airport_daily_meteorology_1995_2024.csv",
    ])
    for path in candidates:
        if os.path.exists(path):
            return os.path.abspath(path)
    raise FileNotFoundError(f"Could not locate Kathmandu dataset CSV in
    candidates: {candidates}")
```

data_processing.py > ...

```python
def audit_raw_dataset(csv_path=None):
    """
    Audits raw daily NASA POWER / MERRA-2 observations (1995-2024) for data
    quality,
    continuity, missingness, sentinel values, and variable summary
    statistics.
    """
    resolved_path = resolve_dataset_path(csv_path)
    df = pd.read_csv(resolved_path)
    dates = pd.to_datetime(df["date"])
    expected_dates = pd.date_range("1995-01-01", "2024-12-31", freq="D")

    raw_vars = [
        "PRECTOTCORR", "T2M", "T2M_MAX", "T2M_MIN", "T2MDEW",
        "RH2M", "QV2M", "PS", "WS10M", "WS10M_MAX",
        "WD10M", "ALLSKY_SFC_SW_DWN", "TS",
    ]

    train_raw = df[dates.dt.year <= 2018].copy()
    train_post = df.iloc[7:][dates.iloc[7:].dt.year <= 2018].copy()

    var_stats = {}
    for col in raw_vars:
        if col == "WD10M":
            s_pre = train_raw.loc[train_raw["WD10M"] >= 0, "WD10M"]
            s_post = train_post.loc[train_post["WD10M"] >= 0, "WD10M"]
        else:
            s_pre = train_raw[col]
            s_post = train_post[col]
        var_stats[col] = {
            "train_prelag_mean": round(float(s_pre.mean()), 4),
            "train_prelag_std": round(float(s_pre.std(ddof=1)), 4),
            "train_postlag_mean": round(float(s_post.mean()), 4),
            "train_postlag_std": round(float(s_post.std(ddof=1)), 4),
            "train_min": round(float(s_post.min()), 4),
            "train_median": round(float(s_post.median()), 4),
            "train_iqr": round(float(np.percentile(s_post, 75) - np.
            percentile(s_post, 25)), 4),
            "train_max": round(float(s_post.max()), 4),
        }

    audit_summary = {
        "resolved_csv_path": resolved_path,
        "raw_total_days": int(len(df)),
        "raw_columns_count": int(df.shape[1]),
        "date_start": str(df["date"].min()),
        "date_end": str(df["date"].max()),
```

data_processing.py > ...

```python
def audit_raw_dataset(csv_path=None):
    audit_summary = {
        "is_monotonic_increasing": bool(dates.is_monotonic_increasing),
        "duplicate_date_count": int(dates.duplicated().sum()),
        "missing_calendar_days": int(len(set(expected_dates) - set(dates))),
        "nan_values_total": int(df.isnull().sum().sum()),
        "fill_value_minus999_total": int((df[raw_vars] == -999.0).sum().sum
        ()),
        "negative_precip_days": int((df["PRECTOTCORR"] < 0).sum()),
        "max_precip_mm": float(df["PRECTOTCORR"].max()),
        "max_precip_date": str(df.loc[df["PRECTOTCORR"].idxmax(), "date"]),
        "sep28_2024_precip_mm": float(df.loc[df["date"] == "2024-09-28",
        "PRECTOTCORR"].values[0]),
        "first_7_days_precip_mm": [float(v) for v in df["PRECTOTCORR"].iloc
        [:7].tolist()],
        "wd10m_negative_calm_days_all": int((df["WD10M"] < 0).sum()),
        "wd10m_valid_days_train_prelag": int((train_raw["WD10M"] >= 0).sum
        ()),
        "wd10m_valid_days_train_postlag": int((train_post["WD10M"] >= 0).sum
        ()),
        "wd10m_negative_days_test": int((df[dates.dt.year > 2018]["WD10M"]
        < 0).sum()),
        "variable_training_stats": var_stats,
    }
    return audit_summary


def load_and_preprocess_data(csv_path=None):
    """
    Loads raw NASA POWER daily meteorological data for the Kathmandu
    Airport grid cell,
    encodes negative WD10M calm-wind flags (-8994.0 / <0) via
    wind_calm_flag, and engineers
    22 physically grounded predictors for Task A (same-day atmospheric
    state classification)
    plus strictly lagged (t-1) predictors for Task B (24-hour lead-time
    forecasting).
    """
    resolved_path = resolve_dataset_path(csv_path)
    df = pd.read_csv(resolved_path)
    df["date"] = pd.to_datetime(df["date"])
    df = df.sort_values("date").reset_index(drop=True)

    # 1. Wind direction flag handling (negative values denote calm/
    undefined valley wind direction)
    df["wind_calm_flag"] = (df["WD10M"] < 0).astype(int)
    df["WD10M_clean"] = df["WD10M"].where(df["WD10M"] >= 0, np.nan)
```

data_processing.py > ...

```python
def load_and_preprocess_data(csv_path=None):
    # 2. Antecedent rainfall memory (strictly shifted to t-1..t-7 so day-t
    rain never leaks into X)
    df["precip_lag1"] = df["PRECTOTCORR"].shift(1)
    df["precip_lag2"] = df["PRECTOTCORR"].shift(2)
    df["precip_lag3"] = df["PRECTOTCORR"].shift(3)
    df["precip_roll3"] = df["PRECTOTCORR"].shift(1).rolling(3).sum()
    df["precip_roll7"] = df["PRECTOTCORR"].shift(1).rolling(7).sum()

    # 3. Thermodynamic & synoptic stability indicators
    df["temp_diurnal_range"] = df["T2M_MAX"] - df["T2M_MIN"]
    df["dew_point_depression"] = df["T2M"] - df["T2MDEW"]
    df["pressure_tendency"] = df["PS"] - df["PS"].shift(1)

    # 4. Seasonal cyclical harmonics (capturing the South Asian Summer
    Monsoon cycle)
    doy = df["date"].dt.day_of_year if hasattr(df["date"].dt,
    "day_of_year") else df["date"].dt.dayofyear
    df["sin_doy"] = np.sin(2.0 * np.pi * doy / 365.25)
    df["cos_doy"] = np.cos(2.0 * np.pi * doy / 365.25)

    # 5. Strictly lagged atmospheric predictors (t-1) for Task B 24-hour
    lead-time forecasting
    day_t_vars = [
        "T2M", "T2M_MAX", "T2M_MIN", "temp_diurnal_range",
        "T2MDEW", "dew_point_depression", "RH2M", "QV2M",
        "PS", "pressure_tendency", "WS10M", "WS10M_MAX",
        "ALLSKY_SFC_SW_DWN", "TS", "wind_calm_flag",
    ]
    for col in day_t_vars:
        df[f"{col}_lag1"] = df[col].shift(1)

    # Trim initial 7 rows (1995-01-01 to 1995-01-07) to initialize 7-day
    rolling window without NaNs
    df_clean = df.iloc[7:].copy().reset_index(drop=True)
    return df_clean


def split_and_construct_target(df, train_end_year=2018, percentile=95,
wet_day_only=False):
    """
    Strict chronological train-test split to prevent temporal lookahead
    leakage.
    Both extreme thresholds (all-day P95 = 13.7400 mm/day and wet-day R95p
    = 22.7520 mm/day
    for wet days R >= 1.0 mm/day) are computed STRICTLY on the 1995-2018
    training partition.
    """
```

data_processing.py > ...

```python
def split_and_construct_target(df, train_end_year=2018, percentile=95,
    train_mask = df["date"].dt.year <= train_end_year
    test_mask = df["date"].dt.year > train_end_year

    train_precip = df.loc[train_mask, "PRECTOTCORR"]
    all_day_threshold = float(np.percentile(train_precip, percentile))
    wet_threshold = float(np.percentile(train_precip[train_precip >= 1.0],
    percentile))

    threshold = wet_threshold if wet_day_only else all_day_threshold

    df["target_extreme"] = (df["PRECTOTCORR"] >= all_day_threshold).astype
    (int)
    df["target_extreme_wet95"] = (df["PRECTOTCORR"] >= wet_threshold).astype
    (int)

    target_col = "target_extreme_wet95" if wet_day_only else
    "target_extreme"

    X_train = df.loc[train_mask, FEATURE_COLS_TASK_A].copy()
    y_train = df.loc[train_mask, target_col].copy()

    X_test = df.loc[test_mask, FEATURE_COLS_TASK_A].copy()
    y_test = df.loc[test_mask, target_col].copy()

    dates_train = df.loc[train_mask, "date"].copy()
    dates_test = df.loc[test_mask, "date"].copy()

    train_wet_days = int((df.loc[train_mask, "PRECTOTCORR"] >= 1.0).sum())
    test_wet_days = int((df.loc[test_mask, "PRECTOTCORR"] >= 1.0).sum())
    train_wet_ext = int(df.loc[train_mask, "target_extreme_wet95"].sum())
    test_wet_ext = int(df.loc[test_mask, "target_extreme_wet95"].sum())

    split_info = {
        "train_start": str(dates_train.min().date()),
        "train_end": str(dates_train.max().date()),
        "test_start": str(dates_test.min().date()),
        "test_end": str(dates_test.max().date()),
        "threshold_mm_exact": float(all_day_threshold),
        "threshold_mm": round(all_day_threshold, 2),
        "wet_day_r95p_threshold_mm_exact": float(wet_threshold),
        "wet_day_r95p_threshold_mm": round(wet_threshold, 2),
        "percentile": percentile,
        "raw_total_samples": 10958,
        "post_lag_total_samples": int(len(df)),
        "train_samples": int(len(y_train)),
        "test_samples": int(len(y_test)),
```

data_processing.py > ...

```python
def split_and_construct_target(df, train_end_year=2018, percentile=95,
    split_info = {
        "train_positives": int(df.loc[train_mask, "target_extreme"].sum()),
        "train_negatives": int((df.loc[train_mask, "target_extreme"] == 0).
        sum()),
        "train_prevalence": float(df.loc[train_mask, "target_extreme"].mean
        ()),
        "test_positives": int(df.loc[test_mask, "target_extreme"].sum()),
        "test_negatives": int((df.loc[test_mask, "target_extreme"] == 0).sum
        ()),
        "test_prevalence": float(df.loc[test_mask, "target_extreme"].mean
        ()),
        "train_wet_days_ge_1mm": train_wet_days,
        "test_wet_days_ge_1mm": test_wet_days,
        "train_wet95_positives": train_wet_ext,
        "train_wet95_prevalence_all_days": float(train_wet_ext / len
        (y_train)),
        "train_wet95_prevalence_wet_days": float(train_wet_ext / max(1,
        train_wet_days)),
        "test_wet95_positives": test_wet_ext,
        "test_wet95_prevalence_all_days": float(test_wet_ext / len(y_test)),
        "test_wet95_prevalence_wet_days": float(test_wet_ext / max(1,
        test_wet_days)),
        "feature_names": FEATURE_COLS_TASK_A,
        "feature_names_task_b": FEATURE_COLS_TASK_B,
    }

    return X_train, y_train, X_test, y_test, dates_train, dates_test,
    split_info


if __name__ == "__main__":
    base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    results_dir = os.path.join(base_dir, "results")
    os.makedirs(results_dir, exist_ok=True)

    audit = audit_raw_dataset()
    df_clean = load_and_preprocess_data()
    X_tr, y_tr, X_te, y_te, d_tr, d_te, split_info =
    split_and_construct_target(df_clean)

    with open(os.path.join(results_dir, "dataset_split_info.json"), "w",
    encoding="utf-8") as f:
        json.dump(split_info, f, indent=2)

    print("=" * 80)
    print("DATA PROCESSING & AUDIT VERIFICATION (src/data_processing.py)")
    print("=" * 80)
```

data_processing.py > ...

```python
    print(f"Resolved Dataset Path : {audit['resolved_csv_path']}")
    print(f"Raw Observations      : {audit['raw_total_days']:,} days ({audit
    ['date_start']} to {audit['date_end']})")
    print(f"Missing / NaN / -999  : NaN={audit['nan_values_total']}, -999=
    {audit['fill_value_minus999_total']}, Gaps={audit
    ['missing_calendar_days']}")
    print(f"Post-Lag Observations : {split_info['post_lag_total_samples']
    :,} days (after 7-day rolling window init)")
    print(f"Training Partition    : {split_info['train_samples']:,} days
    ({split_info['train_start']} to {split_info['train_end']})")
    print(f"  - All-Day P95       : {split_info['threshold_mm_exact']:.4f}
    mm/day -> {split_info['train_positives']} extremes ({split_info
    ['train_prevalence']*100:.2f}%)")
    print(f"  - Wet-Day R95p      : {split_info
    ['wet_day_r95p_threshold_mm_exact']:.4f} mm/day -> {split_info
    ['train_wet95_positives']} extremes ({split_info
    ['train_wet95_prevalence_all_days']*100:.2f}%)")
    print(f"Testing Partition     : {split_info['test_samples']:,} days
    ({split_info['test_start']} to {split_info['test_end']})")
    print(f"  - All-Day P95 Ext   : {split_info['test_positives']} extremes
    ({split_info['test_prevalence']*100:.2f}%)")
    print(f"  - Wet-Day R95p Ext  : {split_info['test_wet95_positives']}
    extremes ({split_info['test_wet95_prevalence_all_days']*100:.2f}%)")
    print(f"Engineered Features   : {len(split_info['feature_names'])} Task
    A predictors | {len(split_info['feature_names_task_b'])} Task B
    predictors")
```

B. Model Training and Evaluation Implementation

train_and_evaluate.py > ...

```python
import os
import sys
import json
import time
import warnings
import numpy as np
import pandas as pd

warnings.filterwarnings("ignore")

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA, FastICA
from sklearn.model_selection import TimeSeriesSplit, GridSearchCV
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, average_precision_score, confusion_matrix,
    brier_score_loss, silhouette_score, davies_bouldin_score,
)
from sklearn.inspection import permutation_importance
from scipy.stats import binomtest


def run_mcnemar_tests(y_true, preds_dict):
    """Computes paired McNemar chi-square statistics (with continuity
    correction) and exact binomial p-values."""
    yt = np.asarray(y_true, dtype=int)
    ext_mask = yt == 1
    pairs = [
        ("Random Forest", "k-Nearest Neighbours"),
        ("Logistic Regression", "k-Nearest Neighbours"),
        ("Random Forest", "Decision Tree"),
        ("Random Forest", "Support Vector Machine"),
        ("Logistic Regression", "Random Forest"),
    ]
    rows = []
    for m1, m2 in pairs:
        p1 = np.asarray(preds_dict[m1], dtype=int)
        p2 = np.asarray(preds_dict[m2], dtype=int)
        c1 = p1 == yt
        c2 = p2 == yt
```

train_and_evaluate.py > ...

```python
def run_mcnemar_tests(y_true, preds_dict):
        b_all = int((c1 & ~c2).sum())
        c_all = int((~c1 & c2).sum())
        chi2_all = float(((abs(b_all - c_all) - 1.0) ** 2) / max(1, b_all +
        c_all)) if (b_all + c_all) > 0 else 0.0
        pval_all = float(binomtest(min(b_all, c_all), n=max(1, b_all +
        c_all), p=0.5).pvalue) if (b_all + c_all) > 0 else 1.0

        e1 = p1[ext_mask] == 1
        e2 = p2[ext_mask] == 1
        b_ext = int((e1 & ~e2).sum())
        c_ext = int((~e1 & e2).sum())
        chi2_ext = float(((abs(b_ext - c_ext) - 1.0) ** 2) / max(1, b_ext +
        c_ext)) if (b_ext + c_ext) > 0 else 0.0
        pval_ext = float(binomtest(min(b_ext, c_ext), n=max(1, b_ext +
        c_ext), p=0.5).pvalue) if (b_ext + c_ext) > 0 else 1.0

        rows.append({
            "pair": f"{m1} vs. {m2}",
            "b_all": b_all,
            "c_all": c_all,
            "chi2_all": round(chi2_all, 2),
            "p_all": pval_all,
            "b_ext": b_ext,
            "c_ext": c_ext,
            "chi2_ext": round(chi2_ext, 2),
            "p_ext": pval_ext,
        })
    return rows


def perform_unsupervised_analysis(X_train, y_train, feature_names):
    """
    Performs K-Means clustering, PCA, and FastICA dimensionality reduction
    strictly on the
    standardized 1995-2018 training partition (N = 8,759 days, 22 features).
    Records empirical fit and transform runtimes.
    """
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)

    # 1. Optimal K search (k = 2 to 6)
    k_range = list(range(2, 7))
    inertias = []
    sil_scores_full = []
    sil_scores_sub = []
    db_scores = []
```

train_and_evaluate.py > ...

```python
def perform_unsupervised_analysis(X_train, y_train, feature_names):
    for k in k_range:
        km = KMeans(n_clusters=k, random_state=42, n_init=10)
        km.fit(X_train_scaled)
        inertias.append(float(km.inertia_))
        sil_full = float(silhouette_score(X_train_scaled, km.labels_,
        sample_size=None, random_state=42))
        sil_sub = float(silhouette_score(X_train_scaled, km.labels_,
        sample_size=5000, random_state=42))
        db = float(davies_bouldin_score(X_train_scaled, km.labels_))
        sil_scores_full.append(sil_full)
        sil_scores_sub.append(sil_sub)
        db_scores.append(db)

    # Fit K-Means with optimal k=3 and measure runtime
    t0 = time.perf_counter()
    km_optimal = KMeans(n_clusters=3, random_state=42, n_init=10)
    train_clusters = km_optimal.fit_predict(X_train_scaled)
    km_runtime = float(time.perf_counter() - t0)

    cluster_df = pd.DataFrame({
        "cluster": train_clusters,
        "target_extreme": np.asarray(y_train, dtype=int),
    })
    cluster_stats = cluster_df.groupby("cluster").agg(
        total_days=("target_extreme", "count"),
        extreme_events=("target_extreme", "sum"),
        extreme_rate=("target_extreme", "mean"),
    ).reset_index()
    total_ext = float(np.sum(y_train))
    cluster_stats["share_of_extremes"] = cluster_stats["extreme_events"] /
    total_ext

    # Compute unscaled physical centroid means per cluster
    X_train_df = pd.DataFrame(X_train, columns=feature_names)
    X_train_df["cluster"] = train_clusters
    unscaled_centroids = X_train_df.groupby("cluster")[feature_names].mean
    ().to_dict(orient="index")

    # 2. PCA & FastICA Dimensionality Reduction (strictly on X_train_scaled)
    t0 = time.perf_counter()
    pca = PCA(n_components=22, random_state=42)
    pca.fit(X_train_scaled)
    pca_runtime = float(time.perf_counter() - t0)

    t0 = time.perf_counter()
    ica = FastICA(n_components=7, random_state=42, max_iter=500)
    ica_coords = ica.fit_transform(X_train_scaled)
```

train_and_evaluate.py > ...

```python
def perform_unsupervised_analysis(X_train, y_train, feature_names):
    ica_runtime = float(time.perf_counter() - t0)

    ica_kurtosis = [
        float(np.mean((ica_coords[:, j] - np.mean(ica_coords[:, j])) **
        4) / (np.var(ica_coords[:, j]) ** 2) - 3.0)
        for j in range(ica_coords.shape[1])
    ]

    var_explained = [float(v) for v in pca.explained_variance_ratio_]
    eigenvalues = [float(v) for v in pca.explained_variance_]
    cum_var_explained = [float(v) for v in np.cumsum(pca.
    explained_variance_ratio_)]

    pca_loadings = pd.DataFrame(
        pca.components_[:3, :],
        columns=feature_names,
        index=["PC1", "PC2", "PC3"],
    )

    unsupervised_results = {
        "k_range": k_range,
        "inertias": inertias,
        "silhouette_scores": sil_scores_full,
        "silhouette_scores_subsample5000": sil_scores_sub,
        "davies_bouldin_scores": db_scores,
        "optimal_k": 3,
        "cluster_profiles": cluster_stats.to_dict(orient="records"),
        "cluster_centers_scaled": km_optimal.cluster_centers_.tolist(),
        "cluster_centers_unscaled": {str(k): v for k, v in
        unscaled_centroids.items()},
        "pca_eigenvalues": eigenvalues,
        "pca_var_explained": var_explained,
        "pca_cum_var_explained": cum_var_explained,
        "pca_components": pca_loadings.to_dict(),
        "ica_kurtosis": ica_kurtosis,
        "runtimes_sec": {
            "PCA": round(pca_runtime, 4),
            "FastICA": round(ica_runtime, 4),
            "KMeans_k3": round(km_runtime, 4),
        },
    }

    return unsupervised_results, km_optimal, pca, ica, scaler


def run_hyperparameter_tuning(X_train_scaled, y_train, X_train_raw):
    """
```

train_and_evaluate.py > ...

```python
def run_hyperparameter_tuning(X_train_scaled, y_train, X_train_raw):
    tscv = TimeSeriesSplit(n_splits=5)
    X_raw_arr = np.asarray(X_train_raw, dtype=np.float64)
    y_arr = np.asarray(y_train, dtype=int)

    # 1. Logistic Regression (L2-Regularized)
    pipe_lr = Pipeline([
        ("scaler", StandardScaler()),
        ("clf", LogisticRegression(class_weight="balanced", solver="lbfgs",
        max_iter=1000, random_state=42)),
    ])
    param_lr = {"clf__C": [0.01, 0.1, 1.0, 10.0]}
    grid_lr = GridSearchCV(pipe_lr, param_lr, cv=tscv, scoring="f1",
    n_jobs=1)
    grid_lr.fit(X_raw_arr, y_arr)
    t0 = time.perf_counter()
    lr_best_C = grid_lr.best_params_["clf__C"]
    lr_final = LogisticRegression(C=lr_best_C, class_weight="balanced",
    solver="lbfgs", max_iter=1000, random_state=42)
    lr_final.fit(X_train_scaled, y_arr)
    lr_fit_time = float(time.perf_counter() - t0)

    # 2. Decision Tree (CART)
    param_dt = {
        "max_depth": [3, 4, 5, 6, 8],
        "min_samples_split": [10, 20, 50],
        "min_samples_leaf": [5, 10, 20],
    }
    grid_dt = GridSearchCV(
        DecisionTreeClassifier(class_weight="balanced", random_state=42),
        param_dt, cv=tscv, scoring="f1", n_jobs=1,
    )
    grid_dt.fit(X_raw_arr, y_arr)
    t0 = time.perf_counter()
    dt_final = grid_dt.best_estimator_
    dt_final.fit(X_raw_arr, y_arr)
    dt_fit_time = float(time.perf_counter() - t0)

    # 3. Support Vector Machine (RBF Kernel)
    pipe_svm = Pipeline([
        ("scaler", StandardScaler()),
        ("clf", SVC(kernel="rbf", class_weight="balanced",
        probability=True, random_state=42)),
    ])
    param_svm = {"clf__C": [0.5, 1.0, 2.0], "clf__gamma": ["scale"]}
    grid_svm = GridSearchCV(pipe_svm, param_svm, cv=tscv, scoring="f1",
    n_jobs=1)
    grid_svm.fit(X_raw_arr, y_arr)
```

train_and_evaluate.py > ...

```python
def run_hyperparameter_tuning(X_train_scaled, y_train, X_train_raw):
    t0 = time.perf_counter()
    svm_best_C = grid_svm.best_params_["clf__C"]
    svm_final = SVC(C=svm_best_C, gamma="scale", kernel="rbf",
    class_weight="balanced", probability=True, random_state=42)
    svm_final.fit(X_train_scaled, y_arr)
    svm_fit_time = float(time.perf_counter() - t0)

    # 4. Random Forest (Ensemble Bagging)
    param_rf = {
        "n_estimators": [100, 150],
        "max_depth": [4, 6, 8],
        "min_samples_leaf": [5, 10],
    }
    grid_rf = GridSearchCV(
        RandomForestClassifier(class_weight="balanced", random_state=42),
        param_rf, cv=tscv, scoring="f1", n_jobs=1,
    )
    grid_rf.fit(X_raw_arr, y_arr)
    t0 = time.perf_counter()
    rf_final = grid_rf.best_estimator_
    rf_final.fit(X_raw_arr, y_arr)
    rf_fit_time = float(time.perf_counter() - t0)

    # 5. k-Nearest Neighbours (Instance-Based)
    pipe_knn = Pipeline([
        ("scaler", StandardScaler()),
        ("clf", KNeighborsClassifier()),
    ])
    param_knn = {"clf__n_neighbors": [5, 9, 15, 21], "clf__weights":
    ["uniform", "distance"]}
    grid_knn = GridSearchCV(pipe_knn, param_knn, cv=tscv, scoring="f1",
    n_jobs=1)
    grid_knn.fit(X_raw_arr, y_arr)
    t0 = time.perf_counter()
    knn_best_k = grid_knn.best_params_["clf__n_neighbors"]
    knn_best_w = grid_knn.best_params_["clf__weights"]
    knn_final = KNeighborsClassifier(n_neighbors=knn_best_k,
    weights=knn_best_w)
    knn_final.fit(X_train_scaled, y_arr)
    knn_fit_time = float(time.perf_counter() - t0)

    def _get_std(g):
        if hasattr(g, "cv_std_"):
            return float(g.cv_std_)
        return float(g.cv_results_["std_test_score"][g.best_index_])
```

train_and_evaluate.py > ...

```python
def run_hyperparameter_tuning(X_train_scaled, y_train, X_train_raw):
    best_models = {
        "Logistic Regression": (
            lr_final,
            {"C": lr_best_C, "penalty": "l2", "solver": "lbfgs",
            "class_weight": "balanced"},
            float(grid_lr.best_score_),
            _get_std(grid_lr),
            lr_fit_time,
        ),
        "Decision Tree": (
            dt_final,
            {**grid_dt.best_params_, "class_weight": "balanced"},
            float(grid_dt.best_score_),
            _get_std(grid_dt),
            dt_fit_time,
        ),
        "Support Vector Machine": (
            svm_final,
            {"C": svm_best_C, "gamma": "scale", "class_weight": "balanced"},
            float(grid_svm.best_score_),
            _get_std(grid_svm),
            svm_fit_time,
        ),
        "Random Forest": (
            rf_final,
            {**grid_rf.best_params_, "class_weight": "balanced"},
            float(grid_rf.best_score_),
            _get_std(grid_rf),
            rf_fit_time,
        ),
        "k-Nearest Neighbours": (
            knn_final,
            {"n_neighbors": knn_best_k, "weights": knn_best_w},
            float(grid_knn.best_score_),
            _get_std(grid_knn),
            knn_fit_time,
        ),
    }

    return best_models


def select_oof_threshold_cv(X_train_raw, y_train, rf_params=None,
fn_weight=15, fp_weight=1):
    """
```

train_and_evaluate.py > ...

```python
def select_oof_threshold_cv(X_train_raw, y_train, rf_params=None,
    """
    tscv = TimeSeriesSplit(n_splits=5)
    X_arr = np.asarray(X_train_raw, dtype=np.float64)
    y_arr = np.asarray(y_train, dtype=int)

    if rf_params is None:
        rf_params = {"n_estimators": 100, "max_depth": 8,
        "min_samples_leaf": 5}

    oof_rf_probs = []
    oof_lr_probs = []
    oof_targets = []

    for tr_idx, val_idx in tscv.split(X_arr):
        rf_fold = RandomForestClassifier(
            n_estimators=rf_params.get("n_estimators", 100),
            max_depth=rf_params.get("max_depth", 8),
            min_samples_leaf=rf_params.get("min_samples_leaf", 5),
            class_weight="balanced",
            random_state=42,
        )
        rf_fold.fit(X_arr[tr_idx], y_arr[tr_idx])
        oof_rf_probs.append(rf_fold.predict_proba(X_arr[val_idx])[:, 1])

        sc = StandardScaler().fit(X_arr[tr_idx])
        lr_fold = LogisticRegression(C=0.1, class_weight="balanced",
        solver="lbfgs", max_iter=1000, random_state=42)
        lr_fold.fit(sc.transform(X_arr[tr_idx]), y_arr[tr_idx])
        oof_lr_probs.append(lr_fold.predict_proba(sc.transform(X_arr
        [val_idx]))[:, 1])

        oof_targets.append(y_arr[val_idx])

    oof_rf_p = np.concatenate(oof_rf_probs)
    oof_lr_p = np.concatenate(oof_lr_probs)
    oof_y = np.concatenate(oof_targets)

    candidate_taus = [0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.40, 0.45, 0.50,
    0.60, 0.70]
    best_tau_rf = None
    best_loss_rf = float("inf")
    oof_sweep = []

    for t in candidate_taus:
        yp_rf = (oof_rf_p >= t).astype(int)
        tn, fp, fn, tp = confusion_matrix(oof_y, yp_rf).ravel()
        loss_rf = int(fn_weight * fn + fp_weight * fp)
```

train_and_evaluate.py > ...

```
def select_oof_threshold_cv(X_train_raw, y_train, rf_params=None,
        yp_lr = (oof_lr_p >= t).astype(int)
        tn_l, fp_l, fn_l, tp_l = confusion_matrix(oof_y, yp_lr).ravel()
        loss_lr = int(fn_weight * fn_l + fp_weight * fp_l)

        oof_sweep.append({
            "tau": t,
            "oof_rf_loss": loss_rf,
            "oof_rf_fn": int(fn),
            "oof_rf_fp": int(fp),
            "oof_rf_tp": int(tp),
            "oof_rf_tn": int(tn),
            "oof_rf_recall": float(tp / max(1, tp + fn)),
            "oof_rf_precision": float(tp / max(1, tp + fp)),
            "oof_rf_f1": float(2 * tp / max(1, 2 * tp + fp + fn)),
            "oof_lr_loss": loss_lr,
            "oof_lr_fn": int(fn_l),
            "oof_lr_fp": int(fp_l),
            "oof_lr_recall": float(tp_l / max(1, tp_l + fn_l)),
        })
        if loss_rf < best_loss_rf:
            best_loss_rf = loss_rf
            best_tau_rf = t

    return float(best_tau_rf), oof_sweep


def calculate_metrics(y_true, y_pred, y_prob):
    """Computes threshold-dependent and threshold-free classification
    metrics from arrays."""
    cm = confusion_matrix(y_true, y_pred)
    tn, fp, fn, tp = cm.ravel()

    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, zero_division=0)
    rec = recall_score(y_true, y_pred, zero_division=0)
    f1 = f1_score(y_true, y_pred, zero_division=0)
    f1_macro = f1_score(y_true, y_pred, average="macro", zero_division=0)
    spec = tn / (tn + fp) if (tn + fp) > 0 else 0.0
    roc_auc = roc_auc_score(y_true, y_prob)
    pr_auc = average_precision_score(y_true, y_prob)
    brier = brier_score_loss(y_true, y_prob)

    return {
        "Accuracy": float(acc),
        "Precision": float(prec),
        "Recall": float(rec),
        "F1 Score": float(f1),
```

train_and_evaluate.py > ...

```
def calculate_metrics(y_true, y_pred, y_prob):
        "F1_Macro": float(f1_macro),
        "Specificity": float(spec),
        "ROC_AUC": float(roc_auc),
        "PR_AUC": float(pr_auc),
        "Brier_Score": float(brier),
        "TN": int(tn),
        "FP": int(fp),
        "FN": int(fn),
        "TP": int(tp),
        "Event_Prevalence": float(np.mean(y_true)),
    }


def bootstrap_confidence_intervals(y_true, y_pred, y_prob,
n_bootstrap=1000, block_length=7, seed=42):
    """
    Computes genuine 95% circular moving-block bootstrap confidence
    intervals across
    n_bootstrap=1,000 resamples with block_length=7 days (matching the
    7-day synoptic
    persistence window) to preserve temporal autocorrelation. When
    block_length=1,
    reduces to standard i.i.d. bootstrap.
    """
    rng = np.random.RandomState(seed)
    y_true_arr = np.asarray(y_true, dtype=int)
    y_pred_arr = np.asarray(y_pred, dtype=int)
    y_prob_arr = np.asarray(y_prob, dtype=np.float64)
    n = len(y_true_arr)

    boot_metrics = {
        "Accuracy": [], "Precision": [], "Recall": [],
        "F1_Score": [], "ROC_AUC": [], "PR_AUC": [], "Brier_Score": [],
    }

    if block_length <= 1:
        for _ in range(n_bootstrap):
            idx = rng.randint(0, n, size=n)
            yt_s = y_true_arr[idx]
            if len(np.unique(yt_s)) < 2:
                continue
            yp_s = y_pred_arr[idx]
            pr_s = y_prob_arr[idx]
            boot_metrics["Accuracy"].append(accuracy_score(yt_s, yp_s))
            boot_metrics["Precision"].append(precision_score(yt_s, yp_s,
            zero_division=0))
```

train_and_evaluate.py > ...

```
def bootstrap_confidence_intervals(y_true, y_pred, y_prob,
            boot_metrics["ROC_AUC"].append(roc_auc_score(yt_s, pr_s))
            boot_metrics["PR_AUC"].append(average_precision_score(yt_s,
            pr_s))
            boot_metrics["Brier_Score"].append(brier_score_loss(yt_s, pr_s))

    ci_results = {}
    for metric, values in boot_metrics.items():
        ci_lower = float(np.percentile(values, 2.5))
        ci_upper = float(np.percentile(values, 97.5))
        ci_mean = float(np.mean(values))
        ci_results[metric] = {
            "mean": ci_mean,
            "ci_lower": ci_lower,
            "ci_upper": ci_upper,
            "block_length": int(block_length),
            "n_bootstrap": int(n_bootstrap),
            "seed": int(seed),
            "str_format": f"{ci_mean*100:.2f}% [{ci_lower*100:.2f}%,
            {ci_upper*100:.2f}%]",
        }
    return ci_results


if __name__ == "__main__":
    from data_processing import load_and_preprocess_data,
    split_and_construct_target

    df_clean = load_and_preprocess_data()
    X_tr, y_tr, X_te, y_te, d_tr, d_te, split_info =
    split_and_construct_target(df_clean)
    unsup_res, km_m, pca_m, ica_m, scaler_m = perform_unsupervised_analysis
    (X_tr, y_tr, split_info["feature_names"])
    X_tr_s = scaler_m.transform(X_tr)
    X_te_s = scaler_m.transform(X_te)
    models = run_hyperparameter_tuning(X_tr_s, y_tr, X_tr)
    print("=" * 80)
    print("TRAINING & EVALUATION MODULE VERIFICATION (src/
    train_and_evaluate.py)")
    print("=" * 80)
    for name, (mdl, params, cv_f1, cv_std, fit_t) in models.items():
        X_ev = X_te.values if name in ["Decision Tree", "Random Forest"]
        else X_te_s
        yp = mdl.predict(X_ev)
        pr = mdl.predict_proba(X_ev)[:, 1]
        m = calculate_metrics(y_te, yp, pr)
        print(f"{name:<23} | CV F1={cv_f1:.4f}+/-{cv_std:.4f} | Test F1={m
        ['F1 Score']:.4f} | Rec={m['Recall']*100:.2f}% | ROC-AUC={m
```

C. *Visual Analytics and Diagnostic Results*

generate_figures.py > ...

```python
import os
import sys
import json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from matplotlib.gridspec import GridSpec

from data_processing import load_and_preprocess_data,
split_and_construct_target
from sklearn.metrics import (
    roc_curve, precision_recall_curve, confusion_matrix,
    brier_score_loss, roc_auc_score, average_precision_score,
)

plt.rcParams["font.family"] = "DejaVu Sans"
plt.rcParams["font.size"] = 9.5
plt.rcParams["axes.titlesize"] = 10.5
plt.rcParams["axes.labelsize"] = 9.5
plt.rcParams["xtick.labelsize"] = 8.5
plt.rcParams["ytick.labelsize"] = 8.5
plt.rcParams["legend.fontsize"] = 8.2
plt.rcParams["figure.dpi"] = 300
plt.rcParams["savefig.dpi"] = 300

NAVY = "#1B365D"
BLUE = "#2B6CB0"
TEAL = "#0D9488"
CORAL = "#C53030"
AMBER = "#D97706"
PURPLE = "#6B46C1"
SLATE = "#4A5568"


def generate_all_figures():
    base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    fig_dir = os.path.join(base_dir, "figures")
    res_dir = os.path.join(base_dir, "results")
    os.makedirs(fig_dir, exist_ok=True)

    verified_path = os.path.join(res_dir, "verified_metrics.json")
    npz_path = os.path.join(res_dir, "test_predictions.npz")
    if not os.path.exists(verified_path) or not os.path.exists(npz_path):
        from run_experiments import main as run_all_exp
        run_all_exp()
```

generate_figures.py > ...

```python
def generate_all_figures():
    if not os.path.exists(verified_path) or not os.path.exists(npz_path):
        from run_experiments import main as run_all_exp
        run_all_exp()

    with open(verified_path, "r", encoding="utf-8") as f:
        verified = json.load(f)
    d_npz = np.load(npz_path)

    df = load_and_preprocess_data()
    X_train, y_train, X_test, y_test, d_tr, d_te, split_info =
    split_and_construct_target(df)
    unsup_res = verified["unsupervised"]

    #
    ----------------------------------------------------------------------
    -
    # 1. figure_01_location.png: Study location & Kathmandu monthly
    climatology
    #
    ----------------------------------------------------------------------
    -
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11.5, 4.4), gridspec_kw=
    {"width_ratios": [1.05, 1.35]})
    ax1.set_facecolor("#EEF4F8")
    lon_c, lat_c = 85.359, 27.697
    rect = patches.Rectangle(
        (lon_c - 0.3125, lat_c - 0.25), 0.625, 0.50,
        linewidth=2.0, edgecolor=NAVY, facecolor="#90CDF4", alpha=0.38,
        label="MERRA-2 Grid Cell (0.5° × 0.625°)",
    )
    ax1.add_patch(rect)
    theta = np.linspace(0, 2 * np.pi, 200)
    ax1.plot(
        lon_c + 0.14 * np.cos(theta), lat_c + 0.10 * np.sin(theta),
        color=TEAL, linestyle="--", lw=1.6, label="Kathmandu Valley Rim (~2,
        000–2,800 m)",
    )
    ax1.scatter(
        [lon_c], [lat_c], color=CORAL, s=90, zorder=5, marker="^",
        label="Tribhuvan Intl. Airport (VNKT, 1,269.39 m)",
    )
    for name, dx, dy in [
        ("Shivapuri (2,732 m)", 0.0, 0.125),
        ("Phulchowki (2,762 m)", 0.08, -0.115),
        ("Chandragiri (2,551 m)", -0.12, -0.06),
    ]:
        ax1.annotate(name, (lon_c + dx, lat_c + dy), fontsize=7.5,
```

generate_figures.py > ...

```python
def generate_all_figures():
        fontweight="bold", color=SLATE, ha="center")
    ax1.set_xlim(lon_c - 0.45, lon_c + 0.45)
    ax1.set_ylim(lat_c - 0.36, lat_c + 0.36)
    ax1.set_xlabel("Longitude (°E)")
    ax1.set_ylabel("Latitude (°N)")
    ax1.set_title("Kathmandu Airport MERRA-2 Grid Domain",
    fontweight="bold")
    ax1.grid(True, linestyle=":", alpha=0.6)
    ax1.legend(loc="lower left", frameon=True, facecolor="white",
    framealpha=0.9, fontsize=7.5)

    df["month"] = df["date"].dt.month
    m_stats = df.groupby("month").agg(
        mean_rain=("PRECTOTCORR", "mean"),
        ext_count=("target_extreme", "sum"),
    ).reindex(range(1, 13))
    months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug",
    "Sep", "Oct", "Nov", "Dec"]
    x_m = np.arange(1, 13)
    ax2.bar(x_m - 0.18, m_stats["mean_rain"], width=0.36, color=BLUE,
    alpha=0.88, label="Mean Daily Rainfall (mm/day)")
    ax2.set_xlabel("Calendar Month (1995–2024)")
    ax2.set_ylabel("Mean Daily Precipitation (mm/day)", color=BLUE,
    fontweight="bold")
    ax2.set_xticks(x_m)
    ax2.set_xticklabels(months)
    ax2.grid(True, axis="y", linestyle=":", alpha=0.6)
    ax2_t = ax2.twinx()
    ax2_t.bar(x_m + 0.18, m_stats["ext_count"], width=0.36, color=CORAL,
    alpha=0.85, label="Extreme Days ≥ 13.74 mm (Count)")
    ax2_t.set_ylabel("Total Extreme Rainfall Days (Count)", color=CORAL,
    fontweight="bold")
    ax2.set_title("Monthly Rainfall Climatology and Monsoon Extreme
    Concentration", fontweight="bold")
    h1, l1 = ax2.get_legend_handles_labels()
    h2, l2 = ax2_t.get_legend_handles_labels()
    ax2.legend(h1 + h2, l1 + l2, loc="upper left", frameon=True,
    facecolor="white", fontsize=8)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_01_location.png"), dpi=300)
    plt.close()

    #
    ----------------------------------------------------------------------
    -
    # 2. figure_02_tail_distribution.png: Daily rainfall tail & P95 vs R95p
    #
```

generate_figures.py > ...

```python
def generate_all_figures():
    -
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11.5, 4.3))
    tr_rain = df.loc[df["date"].dt.year <= 2018, "PRECTOTCORR"].values
    te_rain = df.loc[df["date"].dt.year > 2018, "PRECTOTCORR"].values
    bins = np.linspace(0, 115, 60)
    ax1.hist(tr_rain, bins=bins, color=BLUE, alpha=0.72, label="Training
    (1995–2018, N=8,759)", edgecolor="white", lw=0.4)
    ax1.hist(te_rain, bins=bins, color=TEAL, alpha=0.68, label="Testing
    (2019–2024, N=2,192)", edgecolor="white", lw=0.4)
    ax1.axvline(13.74, color=CORAL, linestyle="--", lw=2.0, label="All-Day
    P95 = 13.74 mm/day (5.01% Train)")
    ax1.axvline(22.752, color=AMBER, linestyle="-.", lw=2.0, label="Wet-Day
    R95p = 22.75 mm/day (1.69% Train)")
    ax1.set_yscale("log")
    ax1.set_xlabel("Daily Precipitation (mm/day)")
    ax1.set_ylabel("Frequency (Log Scale, Days)")
    ax1.set_title("Right-Skewed Daily Rainfall Distribution (Log-Frequency)
    ", fontweight="bold")
    ax1.grid(True, linestyle=":", alpha=0.6)
    ax1.legend(loc="upper right", frameon=True, facecolor="white",
    fontsize=8)

    r_grid = np.linspace(0.1, 95, 300)
    surv_tr = [np.mean(tr_rain >= r) * 100 for r in r_grid]
    surv_te = [np.mean(te_rain >= r) * 100 for r in r_grid]
    ax2.plot(r_grid, surv_tr, color=BLUE, lw=2.2, label="1995–2018 Training
    Exceedance (%)")
    ax2.plot(r_grid, surv_te, color=CORAL, lw=2.2, linestyle="-",
    label="2019–2024 Holdout Exceedance (%)")
    ax2.axvline(13.74, color=CORAL, linestyle="--", lw=1.6)
    ax2.axvline(22.752, color=AMBER, linestyle="-.", lw=1.6)
    ax2.annotate(
        "P95 = 13.74 mm/day\nTrain: 5.01% | Test: 7.48%",
        xy=(13.74, 7.48), xytext=(24, 14),
        arrowprops=dict(arrowstyle="->", color=CORAL, lw=1.2), fontsize=8,
        fontweight="bold", color=CORAL,
    )
    ax2.annotate(
        "Wet-Day R95p = 22.75 mm/day\nTrain: 1.69% | Test: 2.92%",
        xy=(22.752, 2.92), xytext=(36, 3.8),
        arrowprops=dict(arrowstyle="->", color=AMBER, lw=1.2), fontsize=8,
        fontweight="bold", color=AMBER,
    )
    ax2.set_yscale("log")
    ax2.set_xlabel("Precipitation Threshold r (mm/day)")
    ax2.set_ylabel("Exceedance Probability P(R ≥ r) [%]")
    ax2.set_title("Empirical Tail Exceedance Curve & Recent
```

generate_figures.py > ...

```
def generate_all_figures():
    Intensification", fontweight="bold")
    ax2.grid(True, linestyle=":", alpha=0.6)
    ax2.legend(loc="upper right", frameon=True, facecolor="white",
    fontsize=8)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_02_tail_distribution.png"),
    dpi=300)
    plt.close()

    #
    ----------------------------------------------------------------------
    -
    # 3. figure_03_workflow_cv.png: Chronological split & 5-fold
    TimeSeriesSplit
    #
    ----------------------------------------------------------------------
    -
    tau_cv_star = verified["threshold_cost_sweep"]["tau_cv_star"]
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12.2, 4.8), gridspec_kw=
    {"width_ratios": [1.15, 1.2]})
    ax1.axis("off")
    ax1.set_xlim(0, 10)
    ax1.set_ylim(-0.35, 10.15)
    steps = [
        (5.0, 9.10, "1. Raw NASA POWER MERRA-2 Archive\n(1 Jan 1995 – 31
        Dec 2024, N = 10,958 Days)", "#E2E8F0", NAVY),
        (5.0, 7.36, "2. 7-Day Lag Initialization & Strict t−1 Rolling
        Shift\n(22 Engineered Predictors, N = 10,951 Post-Lag Days)",
        "#DBEAFE", NAVY),
        (5.0, 5.62, "3. Strict Chronological Split at 31 Dec 2018\nTrain:
        1995–2018 (8,759 d) | Holdout Test: 2019–2024 (2,192 d)",
        "#FEF3C7", "#92400E"),
        (5.0, 3.88, "4. Training-Only Quantile Thresholds & In-Fold
        Scaling\n(P95 = 13.74 mm/d, Wet-Day R95p = 22.75 mm/d, Pipeline
        Scaler)", "#D1FAE5", "#065F46"),
        (5.0, 2.14, f"5. 5-Fold Expanding-Window TimeSeriesSplit CV\n
        (Hyperparameter Tuning & Leakage-Free OOF t*CV = {tau_cv_star:.2f})
        ", "#EDE9FE", "#5B21B6"),
        (5.0, 0.40, "6. Single Locked Evaluation on 2019–2024 Holdout Set\n
        (164 P95 Extremes, 7-Day Block-Bootstrap CIs, McNemar & Tasks B/C)
        ", "#FEE2E2", "#991B1B"),
    ]
    for x, y, label_txt, fc, ec in steps:
        box = patches.FancyBboxPatch(
            (0.25, y - 0.56), 9.5, 1.12, boxstyle="round,pad=0.08",
            facecolor=fc, edgecolor=ec, linewidth=1.6,
        )
```

generate_figures.py > ...

```
def generate_all_figures():
        ax1.add_patch(box)
        ax1.text(x, y, label_txt, ha="center", va="center", fontsize=7.8,
        fontweight="bold", color=ec, linespacing=1.22)
    for i in range(len(steps) - 1):
        ax1.annotate(
            "", xy=(5.0, steps[i + 1][1] + 0.64), xytext=(5.0, steps[i][1]
            - 0.64),
            arrowprops=dict(arrowstyle="->", lw=1.7, color=SLATE),
        )
    ax1.set_title("Leakage-Free Preprocessing & Pipeline Workflow",
    fontweight="bold", pad=8)

    n_tr = 8759
    n_te = 2192
    test_sz = n_tr // 6
    y_labels = ["Fold 1", "Fold 2", "Fold 3", "Fold 4", "Fold 5", "Final
    Holdout"]
    for i in range(5):
        tr_e = (i + 1) * test_sz
        val_sz = (n_tr - tr_e) if i == 4 else test_sz
        ax2.barh(i, tr_e, left=0, height=0.55, color=BLUE, alpha=0.85,
        label="In-Fold Train" if i == 0 else "")
        ax2.barh(i, val_sz, left=tr_e, height=0.55, color=AMBER, alpha=0.
        90, label="In-Fold Validation (~1,459 d)" if i == 0 else "")
    ax2.barh(5, n_tr, left=0, height=0.55, color=NAVY, alpha=0.90,
    label="Full Train (1995–2018, 8,759 d)")
    ax2.barh(5, n_te, left=n_tr, height=0.55, color=CORAL, alpha=0.90,
    label="Holdout Test (2019–2024, 2,192 d)")
    ax2.axvline(n_tr, color=CORAL, linestyle="--", lw=1.8)
    ax2.set_yticks(range(6))
    ax2.set_yticklabels(y_labels, fontweight="bold")
    ax2.invert_yaxis()
    ax2.set_xlabel("Chronological Sample Index (Days from 8 Jan 1995 to 31
    Dec 2024)")
    ax2.set_title("Expanding-Window 5-Fold TimeSeriesSplit & Holdout
    Design", fontweight="bold")
    ax2.grid(True, axis="x", linestyle=":", alpha=0.6)
    ax2.legend(loc="lower right", frameon=True, facecolor="white",
    fontsize=7.8)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_03_workflow_cv.png"), dpi=300)
    plt.close()

    #
    ----------------------------------------------------------------------
    -
```

generate_figures.py > ...

```
def generate_all_figures():
    # 4. figure_04_kmeans_diagnostics.png: K-Means Silhouette &
    Davies-Bouldin
    #
    ----------------------------------------------------------------------
    -
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11.2, 4.2))
    k_vals = unsup_res["k_range"]
    inertias = [v / 1000.0 for v in unsup_res["inertias"]]
    sils = unsup_res["silhouette_scores"]
    dbs = unsup_res["davies_bouldin_scores"]

    ax1.plot(k_vals, inertias, marker="o", lw=2.2, color=NAVY, label="WCSS
    Inertia (×10³)")
    ax1.set_xlabel("Number of Clusters (K)")
    ax1.set_ylabel("Within-Cluster Sum of Squares (×10³)", color=NAVY,
    fontweight="bold")
    ax1.set_xticks(k_vals)
    ax1.grid(True, linestyle=":", alpha=0.6)
    ax1_t = ax1.twinx()
    ax1_t.plot(k_vals, sils, marker="s", lw=2.2, linestyle="--",
    color=TEAL, label="Silhouette Score")
    ax1_t.set_ylabel("Mean Silhouette Coefficient", color=TEAL,
    fontweight="bold")
    ax1.axvline(3, color=CORAL, linestyle=":", lw=1.8, label="Selected K =
    3")
    ax1.set_title("WCSS Inertia Elbow & Silhouette Profile Across K ∈ {2..6}
    ", fontweight="bold")
    h1, l1 = ax1.get_legend_handles_labels()
    h2, l2 = ax1_t.get_legend_handles_labels()
    ax1.legend(h1 + h2, l1 + l2, loc="center right", frameon=True,
    facecolor="white", fontsize=8)

    ax2.plot(k_vals, dbs, marker="D", lw=2.2, color=CORAL,
    label="Davies–Bouldin Index (Lower is Better)")
    ax2.scatter([3], [dbs[1]], color=NAVY, s=90, zorder=5)
    ax2.annotate(
        f"Global Minimum at K = 3\n(DB = {dbs[1]:.4f}, Sil = {sils[1]:.4f})
        ",
        xy=(3, dbs[1]), xytext=(3.4, dbs[1] + 0.08),
        arrowprops=dict(arrowstyle="->", color=NAVY, lw=1.3),
        fontsize=8.5, fontweight="bold", color=NAVY,
    )
    ax2.set_xlabel("Number of Clusters (K)")
    ax2.set_ylabel("Davies–Bouldin Index", fontweight="bold")
    ax2.set_xticks(k_vals)
    ax2.set_title("Davies–Bouldin Cluster Separation Index Across K ∈ {2..6}
```

generate_figures.py > ...

```
def generate_all_figures():
    ", fontweight="bold")
    ax2.grid(True, linestyle=":", alpha=0.6)
    ax2.legend(loc="upper left", frameon=True, facecolor="white",
    fontsize=8)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_04_kmeans_diagnostics.png"),
    dpi=300)
    plt.close()

    #
    ----------------------------------------------------------------------
    -
    # 5. figure_05_pca_ica_projections.png: 2D PCA & FastICA projections
    #
    ----------------------------------------------------------------------
    -
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11.5, 4.4))
    P_tr = d_npz["P_tr"]
    S_tr = d_npz["S_tr"]
    y_tr_arr = d_npz["y_train"]
    non_ext = y_tr_arr == 0
    ext = y_tr_arr == 1

    ax1.scatter(P_tr[non_ext, 0], P_tr[non_ext, 1], c="#94A3B8", s=6,
    alpha=0.35, label="Non-Extreme Days (N=8,320)")
    ax1.scatter(P_tr[ext, 0], P_tr[ext, 1], c=CORAL, s=16, alpha=0.80,
    edgecolors="white", linewidth=0.2, label="Extreme Days ≥ 13.74 mm
    (N=439)")
    ax1.set_xlabel(f"PC1: Monsoon Moisture & Thermal Saturation ({unsup_res
    ['pca_var_explained'][0]*100:.2f}%)")
    ax1.set_ylabel(f"PC2: Radiative & Wind Advection ({unsup_res
    ['pca_var_explained'][1]*100:.2f}%)")
    ax1.set_title("2D Principal Component Projection (66.46% Combined
    Variance)", fontweight="bold")
    ax1.grid(True, linestyle=":", alpha=0.6)
    ax1.legend(loc="upper left", frameon=True, facecolor="white",
    fontsize=8)

    ax2.scatter(S_tr[non_ext, 0], S_tr[non_ext, 1], c="#94A3B8", s=6,
    alpha=0.35, label="Non-Extreme Days (N=8,320)")
    ax2.scatter(S_tr[ext, 0], S_tr[ext, 1], c=PURPLE, s=16, alpha=0.80,
    edgecolors="white", linewidth=0.2, label="Extreme Days ≥ 13.74 mm
    (N=439)")
    ax2.set_xlabel("Independent Component 1 (IC1: Non-Gaussian Moisture
    Pulse)")
    ax2.set_ylabel("Independent Component 2 (IC2: Convective Instability
    Mode)")
```

generate_figures.py > ...

```
def generate_all_figures():
    ax2.set_title("2D FastICA Non-Gaussian Source Projection (m = 7
    Subspace)", fontweight="bold")
    ax2.grid(True, linestyle=":", alpha=0.6)
    ax2.legend(loc="upper left", frameon=True, facecolor="white",
    fontsize=8)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_05_pca_ica_projections.png"),
    dpi=300)
    plt.close()

    #
    ----------------------------------------------------------------------
    -
    # 6. figure_06_pca_scree_kmeans_radar.png: Combined PCA Scree & K-Means
    Radar
    #
    ----------------------------------------------------------------------
    -
    fig = plt.figure(figsize=(11.8, 4.5))
    gs = GridSpec(1, 2, width_ratios=[1.15, 1.0], figure=fig)
    ax1 = fig.add_subplot(gs[0])
    comps = np.arange(1, 16)
    var_pct = np.array(unsup_res["pca_var_explained"][:15]) * 100
    cum_pct = np.array(unsup_res["pca_cum_var_explained"][:15]) * 100
    ax1.bar(comps, var_pct, color=BLUE, alpha=0.85, label="Individual
    Explained Variance (%)")
    ax1.plot(comps, cum_pct, color=CORAL, marker="o", lw=2.2,
    label="Cumulative Explained Variance (%)")
    ax1.axhline(90.0, color=TEAL, linestyle="--", lw=1.6, label="90%
    Retention Threshold (m=7: 91.46%)")
    ax1.axvline(7, color=TEAL, linestyle=":", lw=1.6)
    ax1.set_xticks(comps)
    ax1.set_xlabel("Principal Component Index (m)")
    ax1.set_ylabel("Explained Variance (%)")
    ax1.set_ylim(0, 105)
    ax1.set_title("PCA Eigenvalue Scree Plot & Cumulative Variance
    Retention", fontweight="bold")
    ax1.grid(True, linestyle=":", alpha=0.6)
    ax1.legend(loc="center right", frameon=True, facecolor="white",
    fontsize=8)

    ax2 = fig.add_subplot(gs[1], polar=True)
    radar_feats = ["QV2M", "RH2M", "T2MDEW", "precip_roll7", "T2M",
    "ALLSKY_SFC_SW_DWN", "temp_diurnal_range", "PS"]
    radar_labels = ["QV2M", "RH2M", "T2MDEW", "Roll7 Rain", "T2M",
    "Shortwave", "Diurnal Range", "Pressure"]
```

generate_figures.py > ...

```
def generate_all_figures():
    f_indices = [split_info["feature_names"].index(f) for f in radar_feats]
    angles = np.linspace(0, 2 * np.pi, len(radar_feats), endpoint=False).
    tolist()
    angles += angles[:1]
    c_colors = [BLUE, AMBER, CORAL]
    c_names = [
        "Cluster 0: Dry Winter (0.41% Ext)",
        "Cluster 1: Pre-Monsoon (0.39% Ext)",
        "Cluster 2: Active Monsoon (94.99% Share)",
    ]
    centers = np.array(unsup_res["cluster_centers_scaled"])
    for c_i in range(3):
        vals = centers[c_i, f_indices].tolist()
        vals += vals[:1]
        ax2.plot(angles, vals, color=c_colors[c_i], lw=2.0, label=c_names
        [c_i])
        ax2.fill(angles, vals, color=c_colors[c_i], alpha=0.12)
    ax2.set_xticks(angles[:-1])
    ax2.set_xticklabels(radar_labels, fontsize=8, fontweight="bold")
    ax2.set_title("K-Means (K = 3) Standardized Centroid Profiles",
    fontweight="bold", pad=16)
    ax2.legend(loc="upper right", bbox_to_anchor=(1.28, 1.14),
    frameon=True, facecolor="white", fontsize=7.5)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_06_pca_scree_kmeans_radar.
    png"), dpi=300)
    plt.close()

    #
    ----------------------------------------------------------------------
    -
    # 7. figure_07_roc_curves.png: Classifier ROC curves with random
    baseline
    #
    ----------------------------------------------------------------------
    -
    y_te = d_npz["y_test"]
    model_specs = [
        ("Logistic Regression", d_npz["pr_lr"], d_npz["yp_lr"], CORAL, "-"),
        ("Random Forest", d_npz["pr_rf"], d_npz["yp_rf"], NAVY, "-"),
        ("Support Vector Machine (RBF)", d_npz["pr_svm"], d_npz["yp_svm"],
        TEAL, "--"),
        ("Decision Tree (CART)", d_npz["pr_dt"], d_npz["yp_dt"], AMBER, "-.
        "),
        ("k-Nearest Neighbours (k=5)", d_npz["pr_knn"], d_npz["yp_knn"],
        PURPLE, ":"),
    ]
```

generate_figures.py > ...

```
def generate_all_figures():
    fig, ax = plt.subplots(figsize=(7.2, 4.8))
    for name, pr, _, col, ls in model_specs:
        fpr, tpr, _ = roc_curve(y_te, pr)
        auc_v = roc_auc_score(y_te, pr)
        ax.plot(fpr, tpr, color=col, linestyle=ls, lw=2.2, label=f"{name}
        (ROC-AUC = {auc_v:.4f})")
    ax.plot([0, 1], [0, 1], color=SLATE, linestyle="--", lw=1.2,
    label="Random Classifier Baseline (AUC = 0.5000)")
    ax.set_xlabel("False Positive Rate (1 - Specificity)")
    ax.set_ylabel("True Positive Rate (Recall / Probability of Detection)")
    ax.set_title("Out-of-Sample Receiver Operating Characteristic Curves
    (2019–2024)", fontweight="bold")
    ax.grid(True, linestyle=":", alpha=0.6)
    ax.legend(loc="lower right", frameon=True, facecolor="white",
    fontsize=8.3)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_07_roc_curves.png"), dpi=300)
    plt.close()

    #
    ----------------------------------------------------------------------
    -
    # 8. figure_08_pr_curves.png: Precision-Recall curves with 7.48%
    baseline
    #
    ----------------------------------------------------------------------
    -
    fig, ax = plt.subplots(figsize=(7.2, 4.8))
    for name, pr, _, col, ls in model_specs:
        prec, rec, _ = precision_recall_curve(y_te, pr)
        ap_v = average_precision_score(y_te, pr)
        ax.plot(rec, prec, color=col, linestyle=ls, lw=2.2, label=f"{name}
        (PR-AUC = {ap_v:.4f})")
    base_rate = float(y_te.mean())
    ax.axhline(base_rate, color=SLATE, linestyle="--", lw=1.4,
    label=f"Climatological Prevalence Baseline ({base_rate*100:.2f}%)")
    ax.set_xlabel("Recall (Probability of Detection)")
    ax.set_ylabel("Precision (Positive Predictive Value)")
    ax.set_ylim(0, 1.02)
    ax.set_title("Out-of-Sample Precision-Recall Curves on Holdout Set
    (2019–2024)", fontweight="bold")
    ax.grid(True, linestyle=":", alpha=0.6)
    ax.legend(loc="upper right", frameon=True, facecolor="white",
    fontsize=8.3)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_08_pr_curves.png"), dpi=300)
```

generate_figures.py > ...

```
def generate_all_figures():
    # 9. figure_09_confusion_matrices.png: 5-model grid with counts & row %
    #
    ----------------------------------------------------------------------
    -
    perf_map = {r["Model"]: r for r in verified["test_performance"]}
    fig, axes = plt.subplots(1, 5, figsize=(14.5, 3.3))
    ordered_models = [
        ("Logistic Regression", "Logistic Regression", d_npz["yp_lr"]),
        ("Decision Tree", "Decision Tree (CART)", d_npz["yp_dt"]),
        ("Support Vector Machine", "RBF-Kernel SVM", d_npz["yp_svm"]),
        ("Random Forest", "Random Forest", d_npz["yp_rf"]),
        ("k-Nearest Neighbours", "k-Nearest Neighbours", d_npz["yp_knn"]),
    ]
    for idx, (m_key, m_disp, yp) in enumerate(ordered_models):
        ax = axes[idx]
        pm = perf_map[m_key]
        cm = confusion_matrix(y_te, yp)
        cm_pct = cm.astype(float) / cm.sum(axis=1, keepdims=True) * 100.0
        ax.imshow(cm_pct, cmap="Blues", vmin=0, vmax=100)
        title_str = f"{m_disp}\n(Rec: {pm['Recall']*100:.2f}%, F1: {pm
        ['F1_Score']:.4f})"
        ax.set_title(title_str, fontsize=8.6, fontweight="bold")
        ax.set_xticks([0, 1])
        ax.set_yticks([0, 1])
        ax.set_xticklabels(["Pred 0", "Pred 1"], fontsize=8)
        ax.set_yticklabels(["Actual 0\n(2,028)", "Actual 1\n(164)"] if idx
        == 0 else ["", ""], fontsize=8)
        for r in range(2):
            for c in range(2):
                val_cnt = cm[r, c]
                val_p = cm_pct[r, c]
                lbl = [["TN", "FP"], ["FN", "TP"]][r][c]
                txt_col = "white" if val_p > 55 else NAVY
                ax.text(
                    c, r, f"{lbl}: {val_cnt}\n({val_p:.2f}%)",
                    ha="center", va="center", color=txt_col, fontsize=8.2,
                    fontweight="bold",
                )
    plt.suptitle(
        "Holdout Confusion Matrices (2019–2024, N = 2,192 Days): Raw Counts
        and Row-Normalized Percentages",
        fontsize=10.5, fontweight="bold", y=1.02,
    )
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_09_confusion_matrices.png"),
    dpi=300, bbox_inches="tight")
```

generate_figures.py > ...

```
def generate_all_figures():
    -
    # 10. figure_10_feature_attribution.png: Multi-panel feature attribution
    #
    ----------------------------------------------------------------------
    -
    feat_df = pd.read_csv(os.path.join(res_dir, "feature_importances.csv")).
    head(10).iloc[::-1]
    fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(13.2, 4.4),
    sharey=True)
    y_pos = np.arange(len(feat_df))

    ax1.barh(y_pos, feat_df["RF_Gini_Importance"] * 100, height=0.38,
    label="Random Forest Gini (%)", color=NAVY)
    ax1.barh(y_pos - 0.38, feat_df["DT_Gini_Importance"] * 100, height=0.
    38, label="Decision Tree Gini (%)", color=AMBER, alpha=0.85)
    ax1.set_yticks(y_pos)
    ax1.set_yticklabels(feat_df["Feature"], fontweight="bold")
    ax1.set_xlabel("Impurity Importance (%)")
    ax1.set_title("Tree Gini Importance (RF vs. CART)", fontweight="bold")
    ax1.grid(True, axis="x", linestyle=":", alpha=0.6)
    ax1.legend(loc="lower right", frameon=True, facecolor="white",
    fontsize=7.8)

    bar_cols = [CORAL if v > 0 else BLUE for v in feat_df["LR_Coefficient"]]
    ax2.barh(y_pos, feat_df["LR_Coefficient"], height=0.55, color=bar_cols,
    alpha=0.88)
    ax2.axvline(0, color=SLATE, lw=1.0)
    ax2.set_xlabel("Standardized Log-Odds Weight (β)")
    ax2.set_title("Logistic Regression Standardized Coefficients",
    fontweight="bold")
    ax2.grid(True, axis="x", linestyle=":", alpha=0.6)

    ax3.barh(y_pos, feat_df["RF_Permutation_AUC_Drop"], height=0.55,
    color=TEAL, alpha=0.88)
    ax3.set_xlabel("Holdout Permutation ΔROC-AUC")
    ax3.set_title("Test-Set Permutation Importance (ΔAUC)",
    fontweight="bold")
    ax3.grid(True, axis="x", linestyle=":", alpha=0.6)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_10_feature_attribution.png"),
    dpi=300)
    plt.close()

    #
    ----------------------------------------------------------------------
    -
```

generate_figures.py > ...

```
def generate_all_figures():
    # 11. figure_11_calibration_reliability.png: Reliability curves & Brier
    #
    ----------------------------------------------------------------------
    -
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11.4, 4.3), gridspec_kw=
    {"width_ratios": [1.25, 1.0]})
    ax1.plot([0, 1], [0, 1], "k--", lw=1.3, label="Perfect Calibration
    (Diagonal)")
    for name, pr, _, col, ls in model_specs:
        bins_cal = np.linspace(0, 1, 8)
        bin_ids = np.digitize(pr, bins_cal) - 1
        mean_p, frac_pos = [], []
        for b in range(len(bins_cal) - 1):
            m = bin_ids == b
            if m.sum() >= 8:
                mean_p.append(pr[m].mean())
                frac_pos.append(y_te[m].mean())
        brier_v = brier_score_loss(y_te, pr)
        ax1.plot(mean_p, frac_pos, marker="o", color=col, linestyle=ls,
        lw=2.0, label=f"{name} (Brier = {brier_v:.4f})")
    ax1.set_xlabel("Mean Predicted Extreme Rainfall Probability")
    ax1.set_ylabel("Observed Empirical Extreme Frequency")
    ax1.set_title("Probability Calibration Reliability Diagram (2019–2024)
    ", fontweight="bold")
    ax1.grid(True, linestyle=":", alpha=0.6)
    ax1.legend(loc="upper left", frameon=True, facecolor="white",
    fontsize=7.8)

    names_b = ["RBF-SVM\n(Platt)", "k-NN\n(k=5)", "Random Forest\n(Bagged)
    ", "Decision Tree\n(CART)", "Logistic Reg.\n(Balanced)"]
    briers = [
        brier_score_loss(y_te, d_npz["pr_svm"]),
        brier_score_loss(y_te, d_npz["pr_knn"]),
        brier_score_loss(y_te, d_npz["pr_rf"]),
        brier_score_loss(y_te, d_npz["pr_dt"]),
        brier_score_loss(y_te, d_npz["pr_lr"]),
    ]
    b_cols = [TEAL, PURPLE, NAVY, AMBER, CORAL]
    bars = ax2.bar(names_b, briers, color=b_cols, alpha=0.88, width=0.52)
    for bar, v in zip(bars, briers):
        ax2.text(bar.get_x() + bar.get_width() / 2, v + 0.004, f"{v:.4f}",
        ha="center", va="bottom", fontsize=8.5, fontweight="bold")
    ax2.set_ylabel("Brier Calibration Loss (Lower is Better)")
    ax2.set_ylim(0, 0.195)
    ax2.set_title("Holdout Brier Score Comparison Across Classifiers",
    fontweight="bold")
```

generate_figures.py > ...

```
def generate_all_figures():
    # 12. figure_12_threshold_cost_sweep.png: RF trade-off & 15:1 cost curve
    #
    ----------------------------------------------------------------------
    sim_df = pd.read_csv(os.path.join(res_dir,
    "operational_threshold_simulation.csv"))
    rf_sub = sim_df[sim_df["Model"] == "Random Forest"].sort_values
    ("Decision_Threshold")
    lr_sub = sim_df[sim_df["Model"] == "Logistic Regression"].sort_values
    ("Decision_Threshold")

    t_sweep = {r["Threshold"]: r for r in verified["threshold_cost_sweep"]
    ["test_sweep"]}
    tau_cv = verified["threshold_cost_sweep"]["tau_cv_star"]
    tau_ph = verified["threshold_cost_sweep"]["tau_posthoc_star"]
    r_cv = t_sweep[tau_cv]
    r_50 = t_sweep[0.50]

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11.6, 4.4))
    ax1.plot(rf_sub["Decision_Threshold"], rf_sub["Recall"] * 100,
    color=TEAL, lw=2.2, marker="o", markersize=4, label="Recall / POD (%)")
    ax1.plot(rf_sub["Decision_Threshold"], rf_sub["Precision"] * 100,
    color=BLUE, lw=2.2, marker="s", markersize=4, label="Precision / PPV (%)")
    ax1.plot(rf_sub["Decision_Threshold"], rf_sub["F1_Score"] * 100,
    color=NAVY, lw=2.2, marker="^", markersize=4, label="F1-Score (%)")
    ax1.axvline(tau_cv, color=CORAL, linestyle="--", lw=1.9, label=f"OOF &
    Test Min τ* = {tau_cv:.2f} (Rec={r_cv['Recall']*100:.2f}%)")
    ax1.axvline(0.50, color=SLATE, linestyle=":", lw=1.6, label=f"Default
    Cutoff τ = 0.50 (Rec={r_50['Recall']*100:.2f}%)")
    ax1.set_xlabel("Decision Probability Threshold (τ)")
    ax1.set_ylabel("Performance Metric (%)")
    ax1.set_title("Random Forest Precision, Recall, and F1 Trade-Off Curve",
    fontweight="bold")
    ax1.grid(True, linestyle=":", alpha=0.6)
    ax1.legend(loc="center left", frameon=True, facecolor="white", fontsize=7.
    6)

    ax2.plot(rf_sub["Decision_Threshold"], rf_sub["Total_Cost_15x"],
    color=NAVY, lw=2.4, marker="o", markersize=4, label="Random Forest
    Holdout 15:1 Loss")
    ax2.plot(lr_sub["Decision_Threshold"], lr_sub["Total_Cost_15x"],
    color=CORAL, lw=2.0, linestyle="--", label="Logistic Regression Holdout
    15:1 Loss")
    ax2.axvline(tau_cv, color=PURPLE, linestyle="-.", lw=1.9,
    label=f"Leakage-Free OOF τ_CV* = {tau_cv:.2f} (Loss = {r_cv
    ['Total_Cost_15x']})")
    ax2.axvline(0.50, color=SLATE, linestyle=":", lw=1.6, label=f"Default
    Cutoff τ = 0.50 (Loss = {r_50['Total_Cost_15x']})")
```

generate_figures.py > ...

```
def generate_all_figures():
    Cutoff τ = 0.50 (Loss = {r_50['Total_Cost_15x']})")
    ax2.scatter([tau_cv, 0.50], [r_cv["Total_Cost_15x"], r_50
    ["Total_Cost_15x"]], color=[PURPLE, SLATE], s=85, zorder=5)
    ax2.set_xlabel("Decision Probability Threshold (τ)")
    ax2.set_ylabel("Asymmetric Cost Loss L(τ) = 15·FN(τ) + 1·FP(τ)")
    ax2.set_title("Asymmetric 15:1 Disaster Cost Curve (OOF τ_CV* vs. Holdout
    Sweep)", fontweight="bold")
    ax2.grid(True, linestyle=":", alpha=0.6)
    ax2.legend(loc="upper left", frameon=True, facecolor="white", fontsize=7.
    8)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_12_threshold_cost_sweep.png"),
    dpi=300)
    plt.close()

    #
    ----------------------------------------------------------------------
    # 13. figure_13_sept2024_case_study.png: 20-30 Sept 2024 Kathmandu flood
    #
    ----------------------------------------------------------------------
    sep_records = verified["september_2024_case_study"]
    d_labels = [pd.to_datetime(r["date"]).strftime("%d Sep") for r in
    sep_records]
    x_s = np.arange(len(d_labels))
    rain_vals = np.array([r["PRECTOTCORR"] for r in sep_records])
    rf_s24_probs = np.array([r["rf_prob_task_a"] for r in sep_records])
    lr_s24_probs = np.array([r["lr_prob_task_a"] for r in sep_records])
    lr24_s24_probs = np.array([r["lr_prob_task_b"] for r in sep_records])

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10.5, 5.2), sharex=True,
    gridspec_kw={"height_ratios": [1.05, 1.1]})
    b_cols = [CORAL if r >= 13.74 else BLUE for r in rain_vals]
    bars = ax1.bar(x_s, rain_vals, color=b_cols, width=0.55, alpha=0.90,
    label="Daily MERRA-2 Rainfall (mm/day)")
    ax1.axhline(13.74, color=CORAL, linestyle="--", lw=1.6, label="All-Day
    Extreme Threshold P95 = 13.74 mm/day")
    ax1.axhline(22.752, color=AMBER, linestyle="-.", lw=1.6, label="Wet-Day
    Threshold R95p = 22.75 mm/day")
    for bar, v in zip(bars, rain_vals):
        ax1.text(bar.get_x() + bar.get_width() / 2, v + 1.8, f"{v:.2f}",
        ha="center", va="bottom", fontsize=7.8, fontweight="bold")
    ax1.set_ylim(0, 108)
    ax1.set_ylabel("Rainfall (mm/day)", fontweight="bold")
    ax1.set_title("Holdout Case Study: 20–30 September 2024 Kathmandu Extreme
    Deluge & Classifier Warnings", fontweight="bold")
    ax1.grid(True, axis="y", linestyle=":", alpha=0.6)
    ax1.legend(loc="upper left", frameon=True, facecolor="white", fontsize=8)
```

generate_figures.py > ...

```
def generate_all_figures():
    ax2.plot(x_s, lr_s24_probs, marker="s", lw=2.3, linestyle="-",
    color=CORAL, label="Logistic Regression (Task A Same-Day)")
    ax2.plot(x_s, rf_s24_probs, marker="o", lw=2.3, linestyle="-",
    color=NAVY, label="Random Forest (Task A Same-Day)")
    ax2.plot(x_s, lr24_s24_probs, marker="^", lw=1.9, linestyle="--",
    color=TEAL, label="Logistic Regression (Task B 24-h Lead Time)")
    ax2.axhline(tau_cv, color=AMBER, linestyle=":", lw=1.6,
    label=f"Cost-Calibrated OOF Cutoff (τ_CV* = {tau_cv:.2f})")
    ax2.axhline(0.50, color=SLATE, linestyle="--", lw=1.4, label="Default
    Cutoff (τ = 0.50)")
    ax2.axhline(0.70, color=PURPLE, linestyle="-.", lw=1.4,
    label="High-Priority Escalation (τ = 0.70)")
    ax2.axvspan(3.6, 8.4, color=CORAL, alpha=0.10, label="Active 5-Day
    Extreme Deluge Window (24–28 Sep)")
    ax2.set_xticks(x_s)
    ax2.set_xticklabels(d_labels, fontweight="bold")
    ax2.set_ylim(0, 1.08)
    ax2.set_xlabel("Date (20–30 September 2024 Holdout Period)")
    ax2.set_ylabel("Predicted Extreme Probability", fontweight="bold")
    ax2.grid(True, linestyle=":", alpha=0.6)
    ax2.legend(loc="upper left", ncol=2, frameon=True, facecolor="white",
    fontsize=7.5)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_13_sept2024_case_study.png"),
    dpi=300)
    plt.close()

    #
    ----------------------------------------------------------------------
    # 14. figure_14_lead_time_wetday_sensitivity.png: Tasks A, B, and C
    #
    ----------------------------------------------------------------------
    sens = verified["sensitivity_tasks"]
    tasks_labels = [
        "Task A: Same-Day\nRandom Forest",
        "Task A: Same-Day\nLogistic Reg.",
        "Task B: 24-h Lead\nLogistic Reg.",
        "Task B: 24-h Lead\nk-NN (k=5)",
        "Task C: Wet R95p\nLogistic Reg.",
    ]
    t_keys = [
        "Task_A_SameDay_RF",
        "Task_A_SameDay_LR",
        "Task_B_24hLeadTime_LR",
        "Task_B_24hLeadTime_kNN",
        "Task_C_WetDayR95p_LR",
    ]
```

generate_figures.py > ...

```
def generate_all_figures():
    rec_vals = [sens[k]["Recall"] * 100 for k in t_keys]
    roc_vals = [sens[k]["ROC_AUC"] * 100 for k in t_keys]
    pr_vals = [sens[k]["PR_AUC"] * 100 for k in t_keys]
    f1_vals = [sens[k]["F1_Score"] * 100 for k in t_keys]

    fig, ax = plt.subplots(figsize=(10.8, 4.4))
    x_t = np.arange(len(tasks_labels))
    w_b = 0.20
    ax.bar(x_t - 1.5 * w_b, rec_vals, width=w_b, color=CORAL, label="Recall /
    POD (%)")
    ax.bar(x_t - 0.5 * w_b, roc_vals, width=w_b, color=NAVY, label="ROC-AUC
    (×100%)")
    ax.bar(x_t + 0.5 * w_b, pr_vals, width=w_b, color=TEAL, label="PR-AUC
    (×100%)")
    ax.bar(x_t + 1.5 * w_b, f1_vals, width=w_b, color=AMBER, label="F1-Score
    (×100%)")
    for i in range(len(x_t)):
        ax.text(x_t[i] - 1.5 * w_b, rec_vals[i] + 1.2, f"{rec_vals[i]:.1f}%",
        ha="center", fontsize=7.5, fontweight="bold")
        ax.text(x_t[i] - 0.5 * w_b, roc_vals[i] + 1.2, f"{roc_vals[i]:.1f}%",
        ha="center", fontsize=7.5, fontweight="bold")
        ax.text(x_t[i] + 0.5 * w_b, pr_vals[i] + 1.2, f"{pr_vals[i]:.1f}%",
        ha="center", fontsize=7.5, fontweight="bold")
        ax.text(x_t[i] + 1.5 * w_b, f1_vals[i] + 1.2, f"{f1_vals[i]:.1f}%",
        ha="center", fontsize=7.5, fontweight="bold")
    ax.set_xticks(x_t)
    ax.set_xticklabels(tasks_labels, fontweight="bold")
    ax.set_ylim(0, 108)
    ax.set_ylabel("Metric Score (%)")
    ax.set_title("Sensitivity Comparison: Task A (Same-Day) vs. Task B (24-h
    Lead Time) vs. Task C (Wet-Day R95p)", fontweight="bold")
    ax.grid(True, axis="y", linestyle=":", alpha=0.6)
    ax.legend(loc="upper right", ncol=2, frameon=True, facecolor="white",
    fontsize=8.5)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_14_lead_time_wetday_sensitivity.
    png"), dpi=300)
    plt.close()
```

D. Operational Dashboard Prototype

